WorkWebs Audio Mixing Ecosystem Tools Documentation

By Larry T. Jost

Version 20260423aa

WorkWebs.com https://github.com/ltjost/WorkWebs-Audio-Mixing-Ecosystem-Tools

Support@WorkWebs.com

https://www.facebook.com/profile.php?id=61557113121231

IMPORTANT:

The tools I have written are available from WorkWebs.com. External components are available as shown below.

Use REAPER 7.25 or later.

There is no warrantee expressed or implied. Use this at your own risk. I am not responsible for any errors, omissions, or defects in this software solution. Be sure to test everything you use and verify its operation is to your satisfaction.

You may rework any of this as long as you do not require payment for your derived product and as long as you reference WorkWebs.com as the source for the original software.

If you would like to send a Thank You gift you may do so from this link or QR code:

https://www.paypal.com/ncp/payment/7W434P69MWXA8

Contents

Important Notes:

- See the readme file for additional information on all the files in the package.
- For REAPER control, use REAPER 7.25 or later to prevent lost routing when saving and reopening REAPER sessions.

Changes made in this release

- **Routing and Samples for combining 1 x32 and 2 WING Racks with the VB Matrix Coconut and REAPER combo.**

Background, Rationale and Links to External Components

After many years of analogue mixing, in 2017 I decided to plan a move to digital. I mostly do live mixing for theater, concerts, or music. I also produce mixes from those events for the people involved. I often need as many as 40-60 inputs. I have been using REAPER for my DAW and for offline mixing. My desire was to use a PC as a mixer so I could use the plugins I wanted and set up my own flows. It was in mid-2018 that I gave up on that quest and bought an X32. I continued my quest to the flexibility I wanted and tried the Waves card and server but found it very restrictive. I looked into LV1 but that was even more constrained in terms of routing and flexibility. I leveraged the wonderful TheatreMix tool by James Holt to make life a lot better when doing theater, but that of course did not free me to control complex routing or a self-designed mixer leveraging the plugins I wanted to use.

I finally bought a Klark-Teknik DN9630 to get 48 channels of input and output and explored how I could turn that into a tool to build a 48 input mixer.

I tried 3 versions of Harrison MixBus over the years and liked the sound a bit but found the routing and time alignment challenging. A 48 input mixer was possible, and I made it work, but it was complex and not very functional. The OSC command control was rather limited, and there really is not a way to set up 100-200 cues like I need for theater and still retain my sanity.

Leveraging the X32 Live card I was able to get bus level output to my PC and mix there and send results back. I was able to use multiple X32s and routing and buses to mix large amounts of channels. I could use OSC and REAPER Web control software to get some practical options. I still felt very constrained, plus the complexity was getting absurd.

I tried GigPerformer and came close to having a large mixer approximating what I wanted, but the usability and scene capability just wasn't there…much like with Harrison MixBus.

Finally I got a powerful enough desktop PC with a Ryzen 9 3900x 12-Core Processor at 3800 Mhz and found it capable of mixing live at 128 samples at 48 Khz with plenty of plugins and tracks. Sadly, it was too cumbersome to carry around to events so I was back to looking for a solution.

MicroCenter had a great deal on a 17 inch laptop with an i7-127000H 2300 Mhz 14 core processor and once I got that, I finally had the hardware needed to achieve my original objectives.

While REAPER is nice, it does not provide enough flexibility in the mixer or other views to really handle live mixing of shows. It most notably lacks multi-touch control. I also needed a way to have lots of cues and to keep my x32 aligned from the same cue system. I also needed a mixer that did not jump to a level if a fader area was clicked on or touched or clicked away from the current position…too risky in live mixing. It also absolutely needed multitouch capability.

I tried MixBus and more OSC programming, but it was not stable enough and the OSC was not robust enough. Similarly with GigPerformer.

I tried TouchOSC, and other tools and just could not get them to operate with the number of faders and widgets I wanted. They bogged down, froze, and became unusable. Simply put, they just would not perform and kept freezing up or crashing.

I finally found Open Stage Control sample code and found it much more capable. I wrote up a json file that gives me what I need. As an added perk, additional copies of the GUI can be executed on a Tablet or other PC via a browser. Using Fully Kiosk Browser I can get the Open Stage Control setup on my Android Tablet for remote control (using a pen to touch things). I like to mix by DCA/VCA, but I also find a desire to have my plugins operate based on a constant input level since changing input level destroys compression, saturation, etc. taking place in the plugins. Because of this, my REAPER layout includes mixing by bus tracks while allowing the input to be separately controlled. Cues are used to dynamically route processed tracks to dynamically named bus tracks for mixing there.

For cue execution, I found LT_Command by Paul Vannatto. Many other options were explored, but they syntax check the OSC commands and are constrained to a specific implementation or do not take files as input or do not allow IP and Port specification at execution time. LT_Command processes files of commands, does not do much by way of syntax checks, and allows IP and Port specification, perfect!

For Cue writing and generation, I also did not want people to have to install software. At the expense of being constrained to Windows PCs, I settled on writing batch CMD scripts to convert friendly cues into OSC. I also wrote a CMD script to select and run the Cues. Everyone on a PC already has CMD program installed!

I also recognize that sometimes I need hot-keys for cue execution, since cues can just be OSC command sets that I want to run from time to time and not really a sequence of Cues that have to run in sequence. I found AutoHotKey can be scripted to enter a cue name and enter it into my cue execution script so any keyboard can be programmed to submit cues for processing…hence making hot keys readily available to run complex OSC scripts.

Then SSL came out with SSL 360 and I was able to get all of the SSL software for $15 per month and decided to try it out.

To enable a person to troubleshoot mic issues, I wrote a REAPER Web Script that can be used to mute and unmute tracks in REAPER. It includes filters to select desired groups of tracks and to display a user specified number of "buttons" per row. Leveraging routing and tracks in REAPER and an output to the X32 unmuting one of these tracks sends the audio to the X32 and out the headphones connected to the x32.

In theater, I usually have an assistant running cues while I mix. I also sometimes also have another person help with troubleshooting so mixing can take place, cues can be advanced, and problems be isolated in a concurrent fashion. This meant I needed a total ecosystem of tools to actually operate. The entire set of tools can also be run by an individual especially for small events and concerts where the dynamics of the cues are much more constrained. Everything had to work together, but finally the potential for all of the pieces existed. I next assembled all the parts and have an operable system.

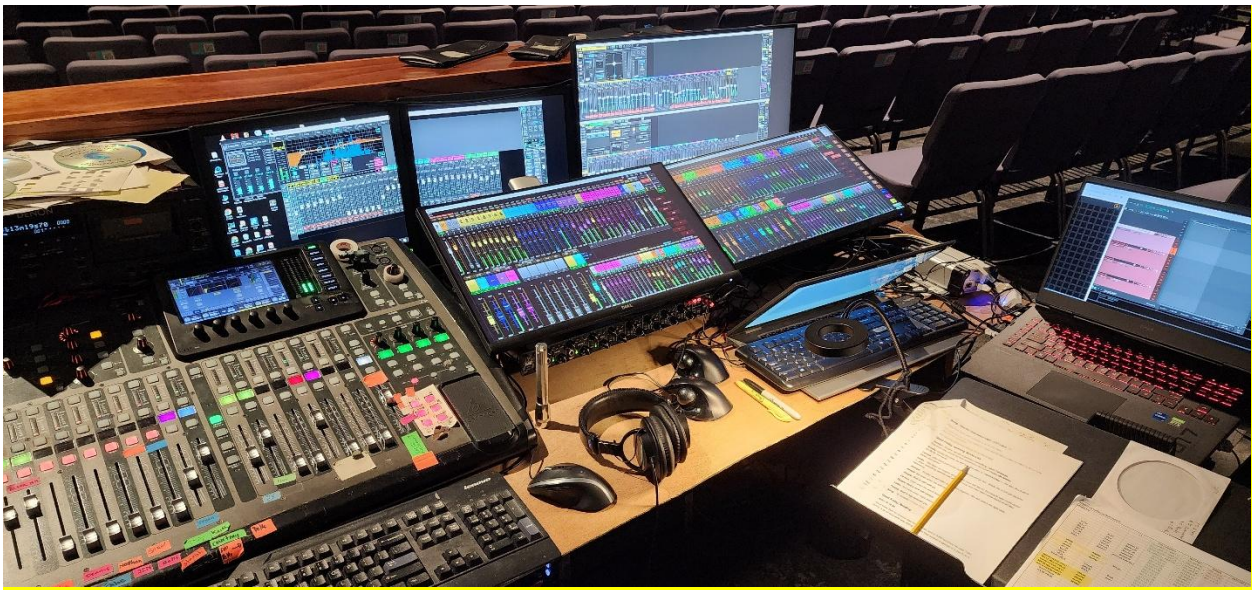
I also created a config for Bitfocus Companion and leveraging the Vicreo Listener can now advance or backup or jump to cues from a web browser! That config is part of this package. The cues can now be advanced on my mixer touch screens).

I also was able to use a 4k screen to view and manipulate SSL360 when I use SSL plugins and/or the REAPER mixer view. Most of that is only needed at rehearsals and once set up is pretty much good to go. But quick access to monitor sends in the REAPER mixer is handy at setup time and the SSL360 view nicely shows compression and gate/expander status that is otherwise hard to access. With large groups of mics, this is a real benefit that I cannot get elsewhere.

I still used an X32 for output routing to amps and sometimes for monitor mixing, and I used an x32 and/or x32 rack and DL16 or 2 or 3 or more of them for connectivity, and that gave me a 48 channel mixer with the routing and plugins I want that can operate fast enough for live mixing, a cue system that can keep everything aligned and flowing, and other tools to help with troubleshooting. This document describes this ecosystem and all of the tools are being made available for anyone to use or otherwise leverage for their own custom solutions. Note that with the DL16 or DL32 that a mixer is optional, but setting gain levels requires access to the stagebox (though there are MIDI commands that can be used to adjust gains using the USB port on the DLxx stage boxes…but I have not explored that…and USB extension of cat5 is easily accomplished…).

Each of these items will be covered in detail in this document. I understand that my solution may not be your solution, but given these tools and these starting points, you can jump in with a good head start into making your own custom solutions! I hope you find this helpful as we forge forward into low-cost and very flexible and capable fully in the box mixing!

But wait, there is more! Recently I encountered a situation where my mixing PC became unstable. It would occasionally crash. Then it got worse and it would constantly crash. Eventually I updated BIOS and it was still unstable. I then reset all BIOS values to their defaults and everything worked great. This, however, took about 2 weeks to figure out. So, I sold my full X32 and my x32 Rack and 2 WING Racks. Now I have configured the 2 WING racks and an X32 to flow data between them via AES50 so I have about a 100 channel mixer with 10 monitor feeds and shared effects between them all, and allowance to route to external hardware. With plenty of track capacity between these devices, I walked away from mixing live with REAPER with stellar results. The plugins in the WING are amazing and the addition of some external effects adds to the flexibility. The only thing missing was a nice Mixing Station config and the ability to use this same type of cue system on the WING…so I set up a Mixing Station Config, and use that alongside of a WING edit session and racked my WINGs together and the results are just as I need, with the cue control I want. It is this WING cue capability that has been added in the 20250811a release and continues in this relea



A 90+ Channel mixer for live mixing is described and provided. Remember, the wing channels are stereo! To allow sound check playback, VB Matrix Coconut merges inputs form the x32 and 2 WING racks and allows recording and playback from REAPER. I also added a Mixing Station layer that allows a mic tech to solo channels and send them to an external headphone amp (old analogue mixer) so that person can do mic repairs and verify them with considerable autonomy.

Implementation Illustration with 1 X32 and 2 WING Racks (not visible in this view) with REAPER for recording and sound check/playback 128 Channel Mixer for Live Mixing

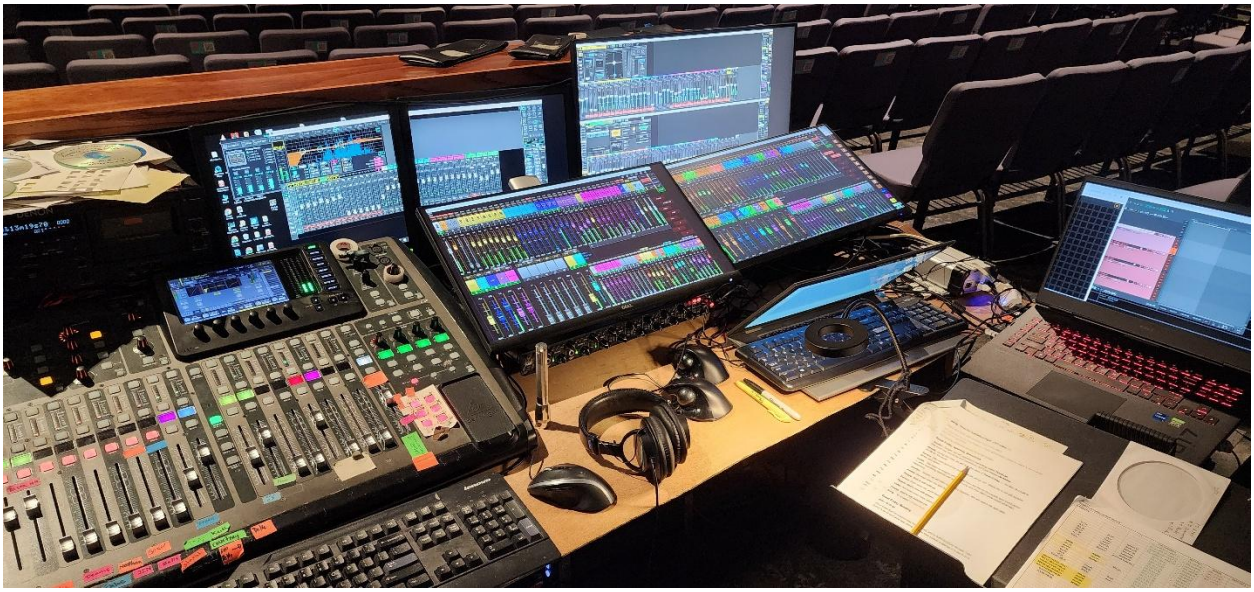


Figure 1 A Sample setup

In this setup is an X32 that exists in this venue and always has the main and monitor etc. outputs in place wired up along with a large set of normally used mics and audio sources. Those were leveraged in this setup and routing to and from the x32 was via AES50. Not shown, but underneath the table were 2 WING racks, AES50 connected to each other and chained to the X32. The X32 connected to the laptop on the right via USB to record and make available playback of up to 32 channels sourced on the X32. It also connects to the first WING Rack via USB to allow 48 channels in/out for recording/playback and connects to the ING Rack number 2 via AES50 and a DN9630 for an additional 48 channels of recording/playback. Using VB Matrix Coconut all of these ins and outs route to REAPER cohesively to allow sound check playback and even later mixing to take place. This is fantastic and they all feed to and send from REAPER as sequential inputs and outputs as though they were all from a single interface! I have seen many people say this input merging was not possible without a Macintosh computer, but now it is available for Windows as well! The Laptop display is used to see the VB Audio Matrix Coconut and REAPER. The 4k monitor top right is connected to the HDMI port of the laptop and it shows two instances of WING Edit…one for each of the WING Racks. Then there are 2 laptops running two touchscreens, one for Mixing Station on WING 1 and one for Mixing Station on WING2. Mouse sharing is provided since this photo was taken via Microsoft Garage Mouse without Borders. It uses network connectivity to allow the mouse to span computers…and screens. There are also a pair of old displays showing the X-Edit for the X32. An old PC is used to run this. The lower right PC with its display folded

down a bit is also running the cue software and a Vicreo Listner with Companion server running on the laptop on the far right of the photo. A Companion web interface is seen on the touchscreen for WING 2 next to Mixing Station to allow cues to be selected using the Cue system provided in this package.

All configs and setups are included in the download of this ecosystem. For the most part, they are set for playback, but the record solution is available by arming in REAPER, using PLAYBACK in x32 routing and using the Main inputs in WING.

This is a 100+ source mixing setup suitable for live mixing! With the added X32 there are approximately 96 available channels without considering the AUXs.

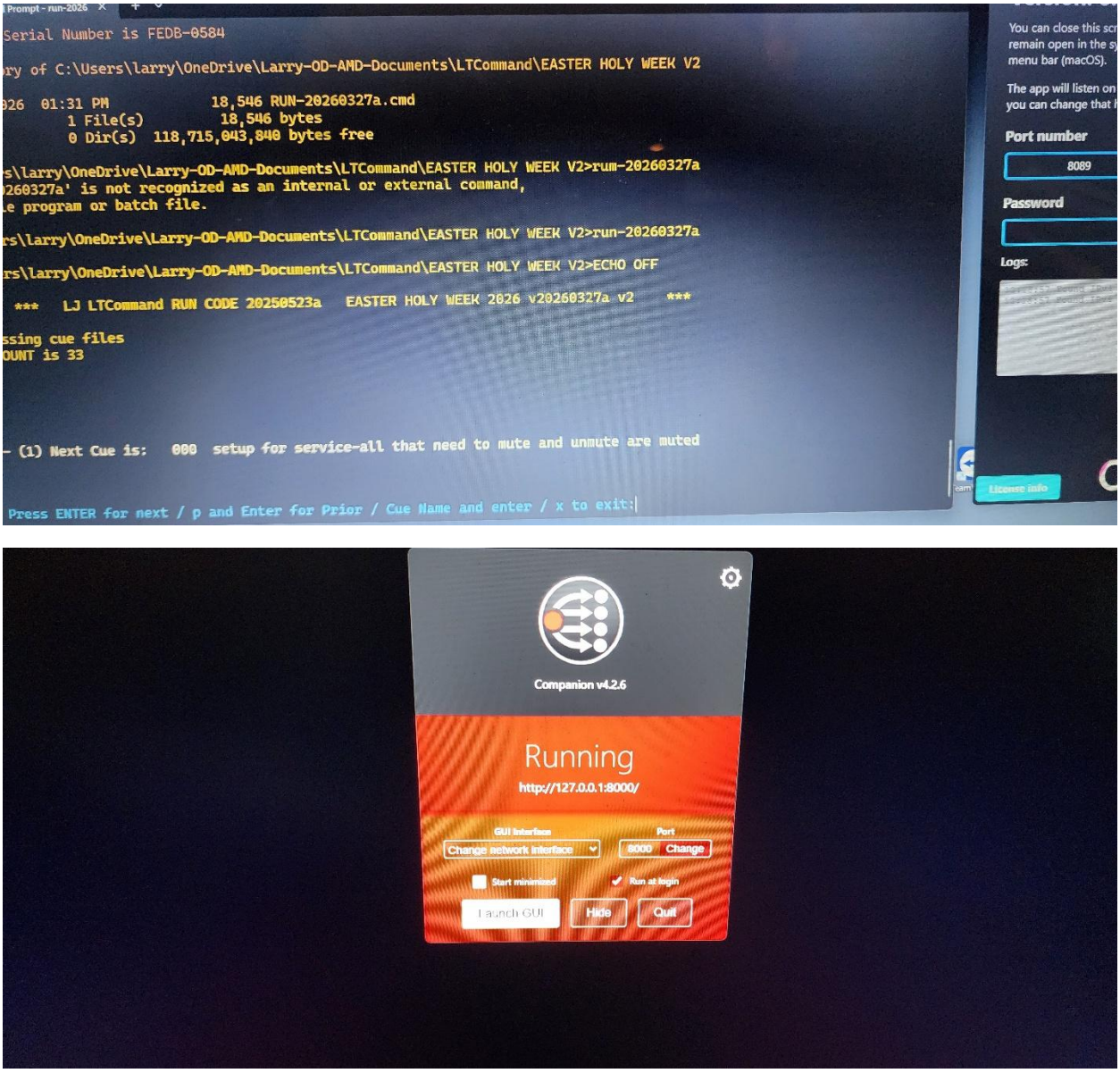


Figure 2 Cues and Vicreo Listener laptop and Companion Screen

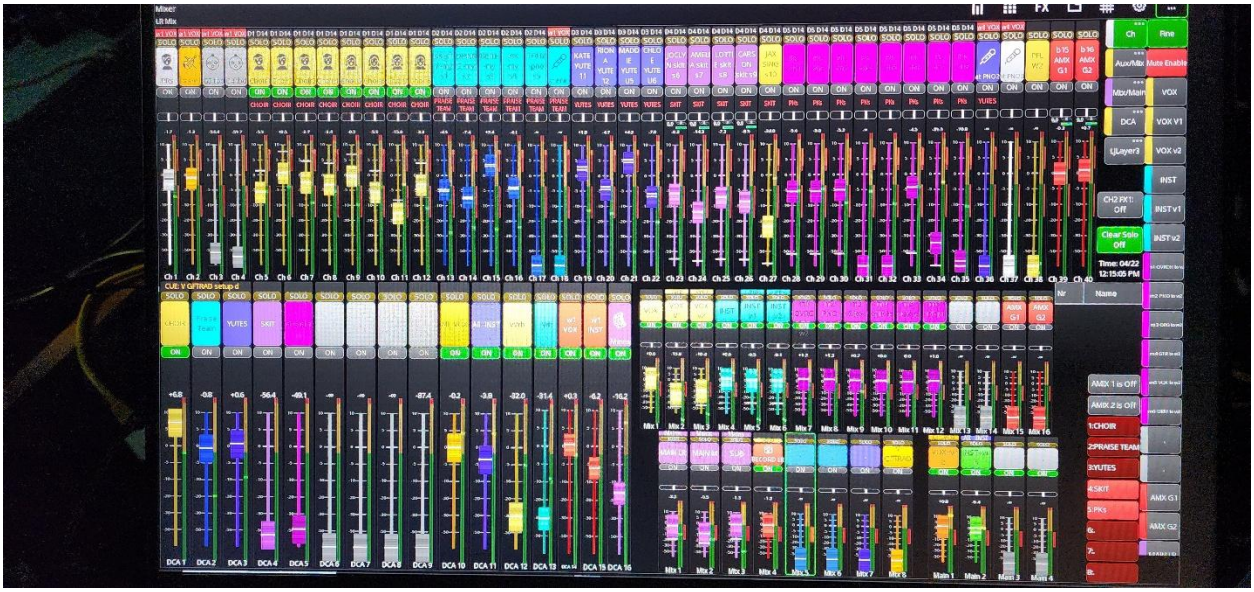


Figure 3 WING 1 Rack Mixing Station session

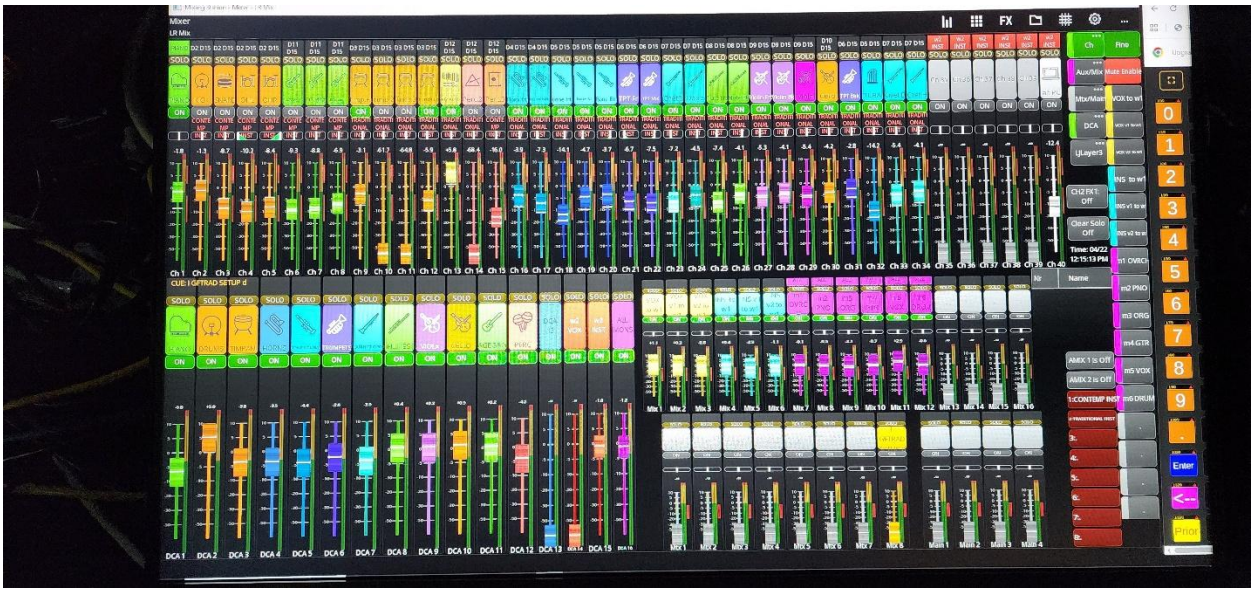


Figure 4 WING2 Mixing Station session with Companion /tablet session on a browser for Cues.

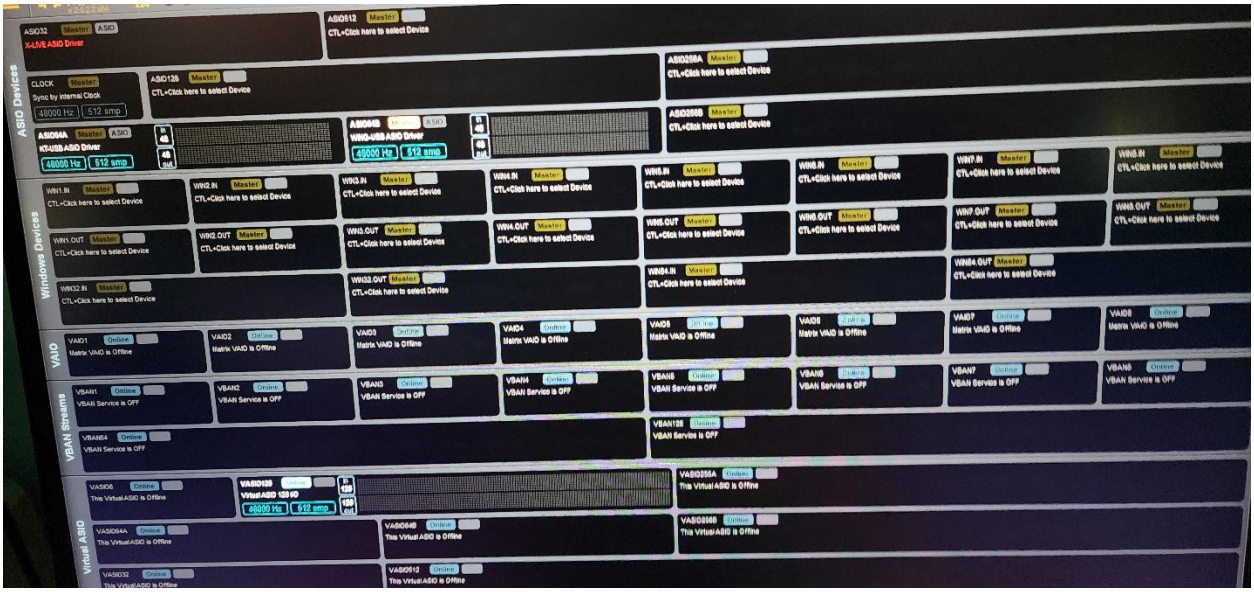


Figure 5 VB Audio Matrix Coconut setup

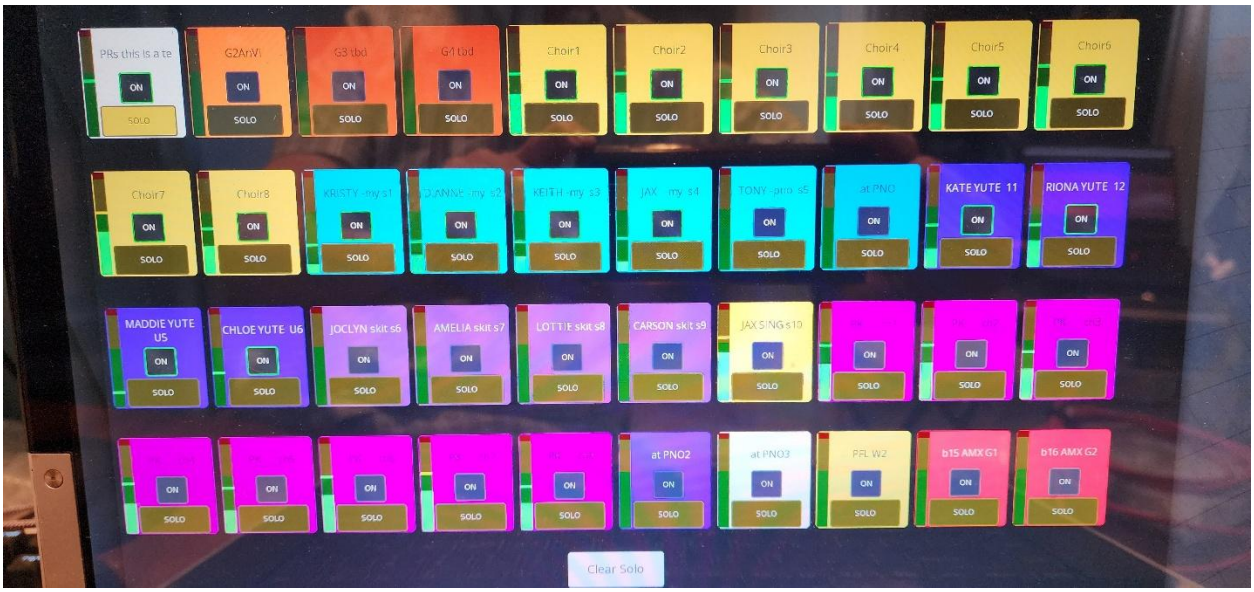


Figure 6 SOLO handling for 40 WING main Channels

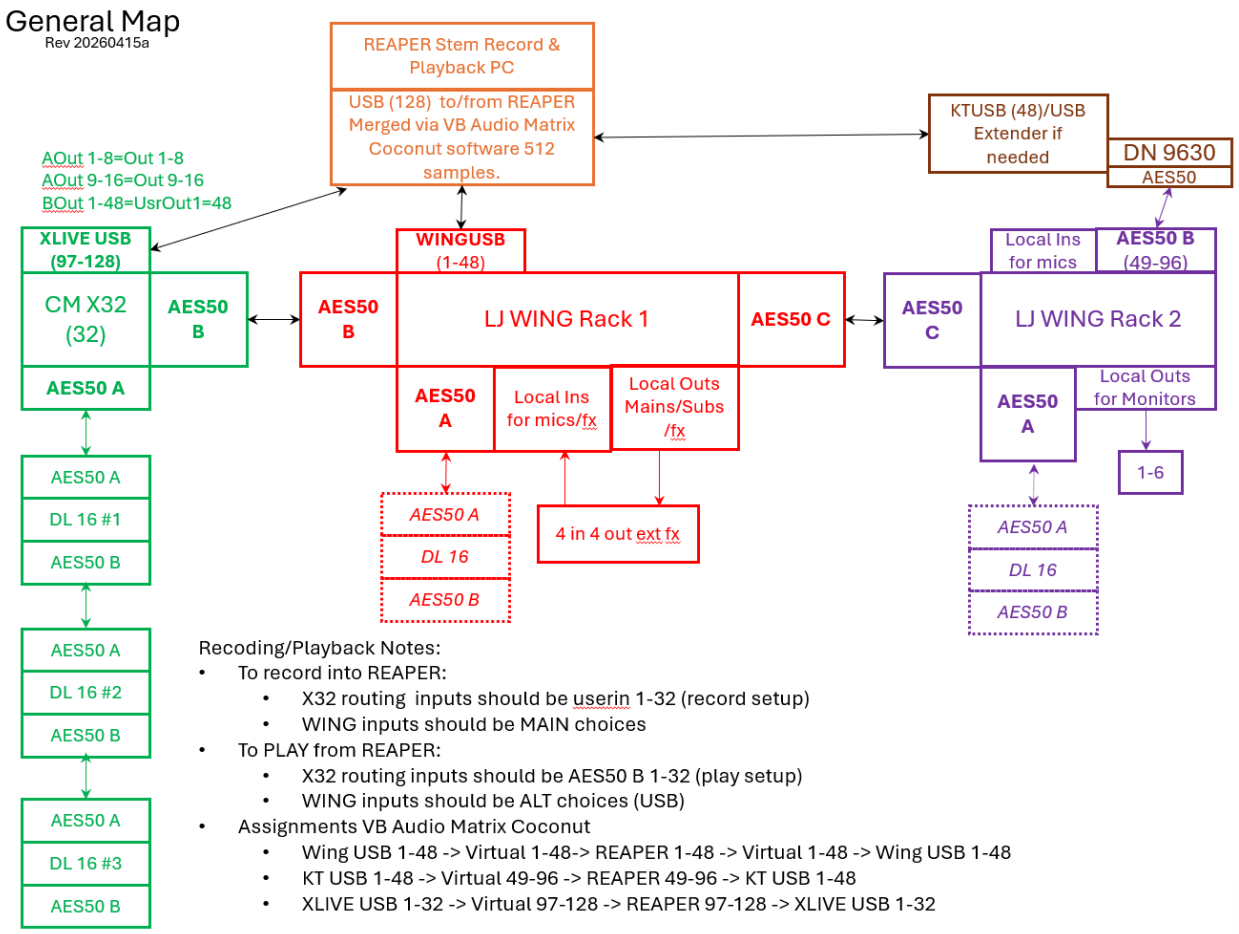


Figure 7 Flow of information between X32, 2 WING racks and REAPER with VB Audio Coconut

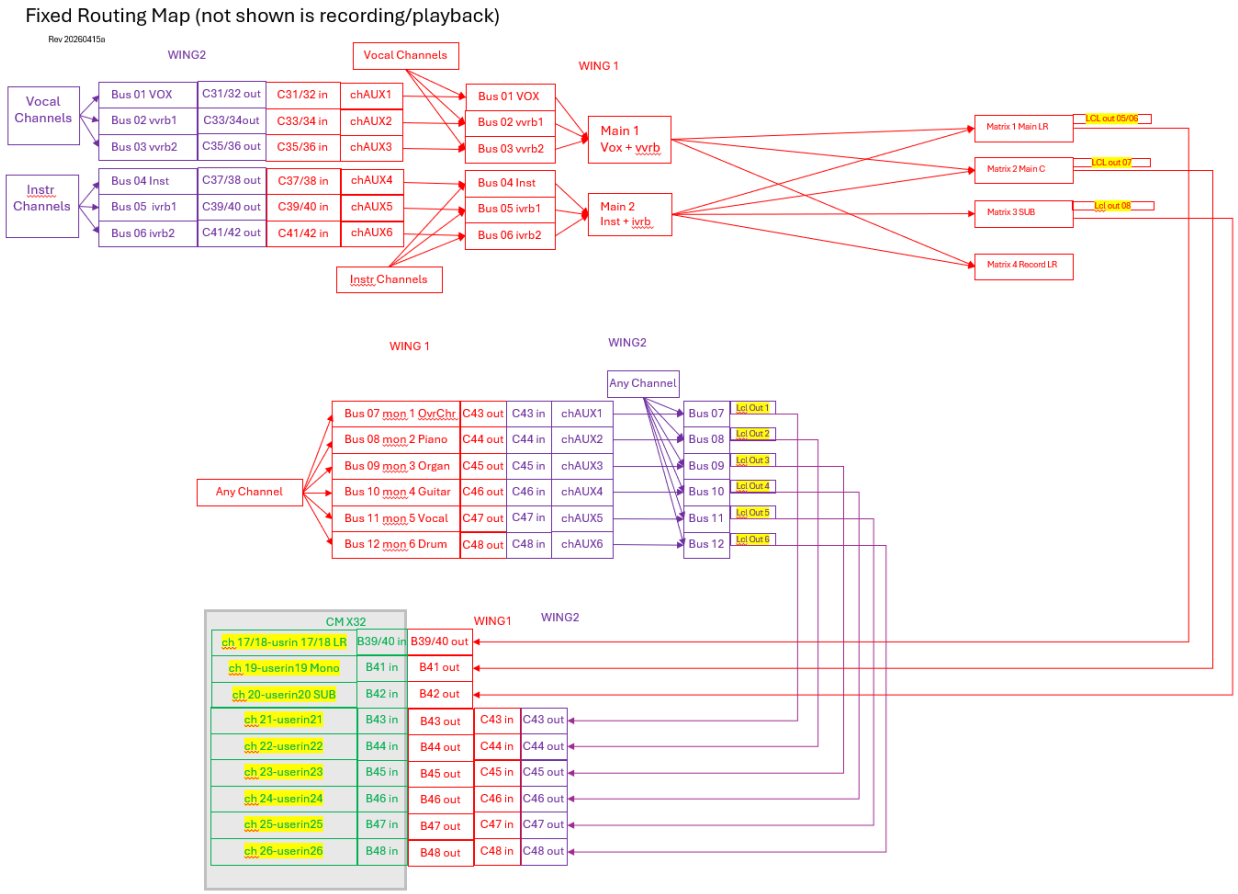


Figure 8 Detailed configuration information for data flow.

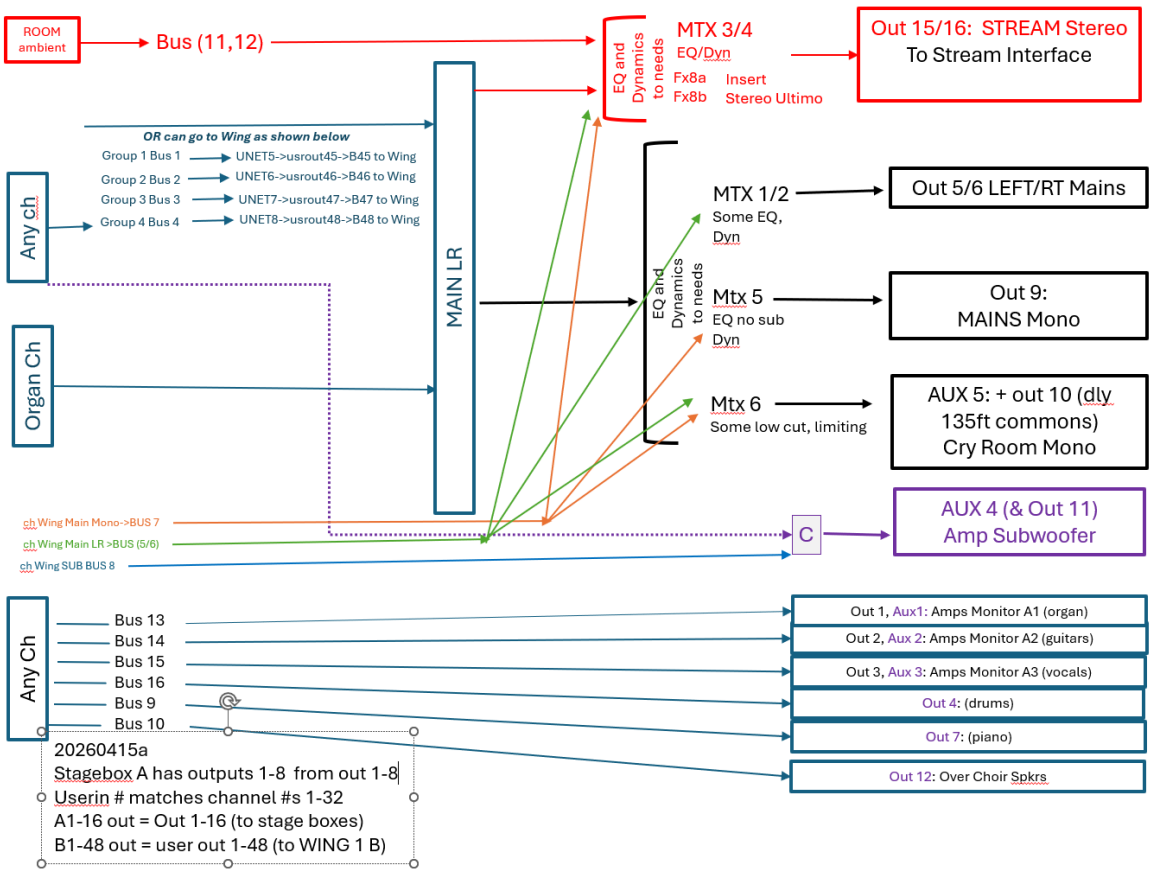


Figure 9 Flow to pre-existing X32 to enable flows without rewiring.

Implementation Illustration with REAPER as the Mixer (an option not used in this release sample)

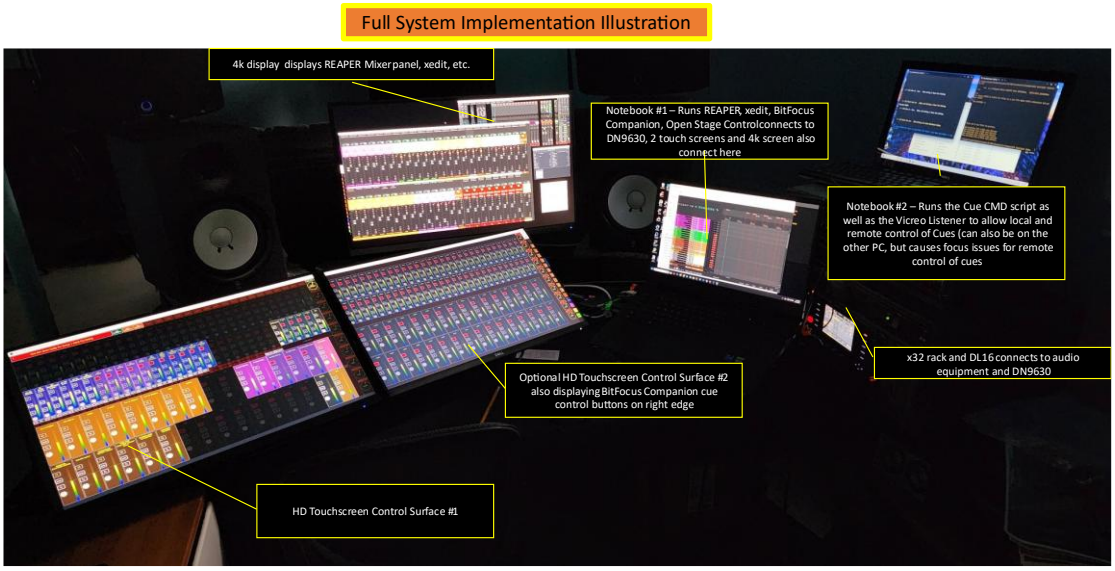
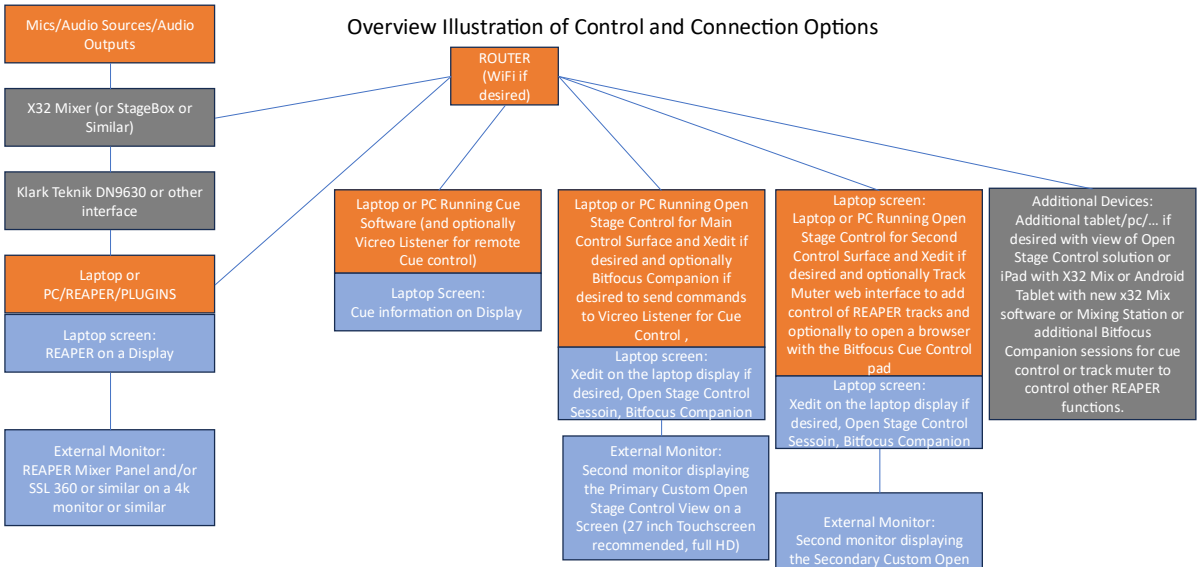


Figure 10 A sample full implementation of the system

Overview of the Ecosystem With REAPER and X32, Mixing in REAPER



Note that the PC running REAPER can also run the Open Stage Control interface/screens and Cues and BitFocus Companion, and xedit but cue execution requires window focus on that window so it is usually best to have the cues executed on a dedicated PC if there are many cues needed.)

Figure 11 High level diagram of the REAPER mixing ecosystem.

The ecosystem high-level is illustrated in this figure. Below is some detail on the solution to provide a high-level view.

- **Audio sources and outputs** need to interconnect to the REAPER instance via some hardware interface. In my case I have an X32 Mixer and Midas DL16 stage box components. Because of this, I use a Klark Teknik DN9630 from the X32 AES50 port to connect 48 channels in and out to REAPER on a PC via a USB2 connection. Note that you should be sure to use the KT driver for the DN9630. Other drivers have given me problems or not worked well at all. You may use any audio interface you want that works for your solution. Note that the stage box is usable without the mixer in this setup, no mixer is needed. The gain and phantom power for the inputs can be directly controlled from the front of the DL16 (or DL32). *There is even a way, I am told, of managing the gains via the USB interface on the DL16 and a MIDI solution, but I have not explored that option. Note that USB2 extenders via Cat5 are readily available so that proximity is optional if the MIDI solution is available.*
- **The PC running REAPER is the processing** focus of this ecosystem. It needs to be powerful enough to handle your selected plugins, number of tracks configured, etc. It is recommended that you only use this PC for this single function to prevent audio disruption, though I have also been able to run the entire set of components form this same PC and mix sound successfully. In my situation I have tested a Ryzen 9 3900x 12-Core Processor at 3800 Mhz in a desktop as well as an i7-127000H 2300 Mhz 14 core processor laptop.
    - I have varied from using the SSL plugin suite for my primary use in this solution, Plugin Alliance bx_console AMEK 9099, or Harrison 32Classic Channel Strip because they are very CPU friendly, and some have ECO mode. In this latest implementation I am using Plugin Alliance's Lindell 80 for most of the work.
    - I have also tested with some UAD, Slate, WAVES and PA strips and they all work fine, but sometimes not with a 128 sample buffer when populating a full 48 tracks running along with SSL 360. SSL 360 can be designated to use your GPU for video processing to save some of its load form hitting your CPU. Similarly for REAPER itself.
    - Your processor power needs to work with your selected plugins and the number of tracks you plan to deploy and the buffer size you can live with.
    - Note that if you do use a mixer, that you will need to route the inputs to the interface and the outputs from REAPER to the mixer outputs. Routing back generally uses channels on the mixer. The remaining channels, however, can also take inputs and can be used to mix your monitors. You just won't be able to select as many inputs for monitoring. I use the User Out and User In on the x32 for routing. The User Out mapping determines what goes to the AES50 that feeds the DN9630 and REAPER. The user in maps from the AES50 from REAPER along with anything I want locally on the X32 and then I use channels on the x32 to mix from there. If I also want inputs to mix monitors direct from the X32 I use User In routing.
    - **SSL360** provides a very nice tool for adjusting EQ, Compression, Gates, etc. and to see them all in one place for quick access for adjustments and was my selected option earlier. I did not mix there, but do use a 4k monitor. I do adjust EQ, dynamics, etc. there, but not faders. The Open Stage Control interface gives me rapid and all-available fader access that I require, especially for rapidly changing musical theater needs. Running SSL360 must take place on the PC running REAPER if you use it.

- **The REAPER Web Interface software** I have written is for track muting and unmuting. I also provide a REAPER Web script that can also raise and lower REAPER fader levels by set increments. In this ecosystem it is employed to unmute PFL tracks in REAPER so they send their output audio to a REAPER output to get picked up by a SOLO selection on the X32 to be fed to headphones. In fact, I usually use a headphones splitter so I and my troubleshooting assistant can both hear things as needed. The REAPER routing has every input send to another track so that I can listen to the pre-effects audio by just unmuting what I want to hear when I want to hear it. The software can filter by a text string in the track name. All the REAPER PFL tracks in REAPER can, for example, have "PFL" in their track names so I can filter on "PFL" and see all of these and only these tracks in the tool. Using no filter I see them all, using some other filter I see what I asked for. Since this is Java Script and a web tool, it can run on any device that has network access to REAPER. Note that you can require a password in REAPER, more on configuring the interface in the details on this tool.
- **There is also a need to have a PC running the Open Stage Control software.** This is pretty light on CPU as well, but I found I get more out of my REAPER PC if I separate this to a PC that is not also running my REAPER instance. I connect one or two 27 inch touchscreens to it and mix from there. Note that the Open Stage Control engine tells you an IP address you can use to also web connect to the same content. This allows mixing the same view and same view in multiple devices concurrently. I sometimes leverage this on a tablet, though it is small, using a pen interface. Note that a full screen kiosk browser on the tablet makes the solution more usable. That link appears elsewhere in this document. Xedit or other software can be run on this PC as well.
- **The Cue PC is also needed.** This is also a low CPU process when running, though it is heavy on CPU when building cues. The Cue process, described fully later, is a two step solution. Cues are written in a human readable form with named references to tracks and English words used to describe actions. These files are processed by a BUILD script to make OSC commands out of them…presently for REAPER or X32. Each Cue is placed in a separate file. Each Cue can be assigned a target IP at run time based on the naming of the cue file so you can control zero or more x32s and zero or more connected REAPER instances from the running of a common set of Cues at the RUN step. Cues are designed to run in sequence, but also can be run by name. You can skip the build step and just build directly in OSC if you'd like. The OSC files are what get run. Using AutoHotKey you can set up key commands to execute Cue Files…so that can be a handy way to run any cue any time you want. They can even have follow cues with or without time delays first and you can run them sequentially or can run cues in separate processes. All of the scripts are written in DOS Command Scripts so only work on a PC. However, no added software is required. Notepad can be used to create and edit the cue files. The BUILD and RUN scripts must be edited by you to define things for your setup. They are also easy to edit if you want to develop your own customizations. If the design of the script changes at some later time, the unified scripts and setup means that what you saved is a complete script solution so later base script versions will not impact their operation. This is another plus for using these types of scripts instead of complied code. A footswitch can also be programmed to trigger cues, though Auto Hot Key may need to be used with them. To keep focus on the RUN script, do not run any other software on this PC. Bitfocus Companion and the Vicreo Listener can also be used to control Cues remotely. A configuration is included with this package.

- ***Note that the tools can be used outside of this complete Ecosystem as well. I just like using all of them together for a holistic solution.***

Track Muter for REAPER

Overview

The Track Muter is a script that runs through the REAPER web interface system. It is included in the package of items provided by WorkWebs.com free of charge and runs through a web browser on a connected device.

Figure 12 Track Muter Web Interface for REAPER

This tool displays the current mute status of REAPER tracks and gives you a button to click/touch to toggle mute/unmute status of a track in REAPER. The area of the display surrounding the mute button is colored to match the color of the track as you specified it in REAPER. The button area is red if the track is muted, white if it is not muted.

Usage

By default it shows all of the tracks in the connected session.

However, you can filter it to track names containing a specific text string. This is done by typing the text string to filter on (case sensitive in the entry area and then right clicking on the text entry field to make the filter active. In the figure above you will see "PFL" in the filter text area.

There is also a field provided for you to enter a number specifying how many buttons to place in each row of the page. To change the value, type in a new number and then right click in the text area where you entered the number to make the change active.

Note that Folders will appear here with an indication that they are folders above the track name to assist usage.

Installation

After you download the package from WorkWebs.com, open your REAPER session and go to Options->Preferences->Control/OSC/Web and click the Add button. Under Control Surface Mode select Web Browser interfaced. Click the User pages button to open the folder into which you will need to copy the Track Muter script. Copy the script to the folder. Then follow the REAPER instructions to select a port for the web server for this script and set up access using reaper.fm if you would like. Apply the settings or click OK to activate it. You may then access the script from any device connected to the same network using either the reaper.fm URL provided or the IP address and port.

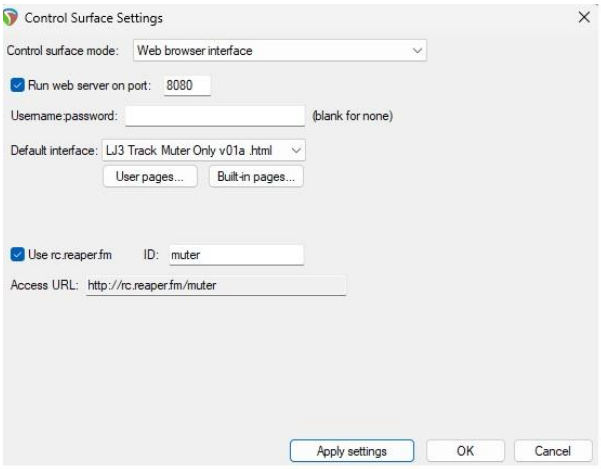


Figure 13Web Script configuration in REAPER

Technical Notes

The script is something you can edit with a text editor if you would like to change things. It is easy to change colors and other things as desired. The released version polls for updates every 1 second (1000 ms). I have found that too frequent of updates can cause slow response and difficulty in using the script. At 1 second it seems to be usable and stable. It does mean there will be a delay in getting the screen updated when you make changes, but the changes are still being transmitted to REAPER in a timely manner, the updated status is independent and is just one once per second. The screen updates are based on this polling, not on your button presses. This allows updates from any device to be reflected every 1 second. The code of this timing is towards the end of the script is looks like this:

```
wwr_start();
wwr_req_recur("TRACK",1000);
```

Track Muter and Volume Adjuster for REAPER

Overview

The Track Muter and Volume Adjuster is a script that runs through the REAPER web interface system. It is included in the package of items provided by WorkWebs.com free of charge and runs through a web browser on a connected device.

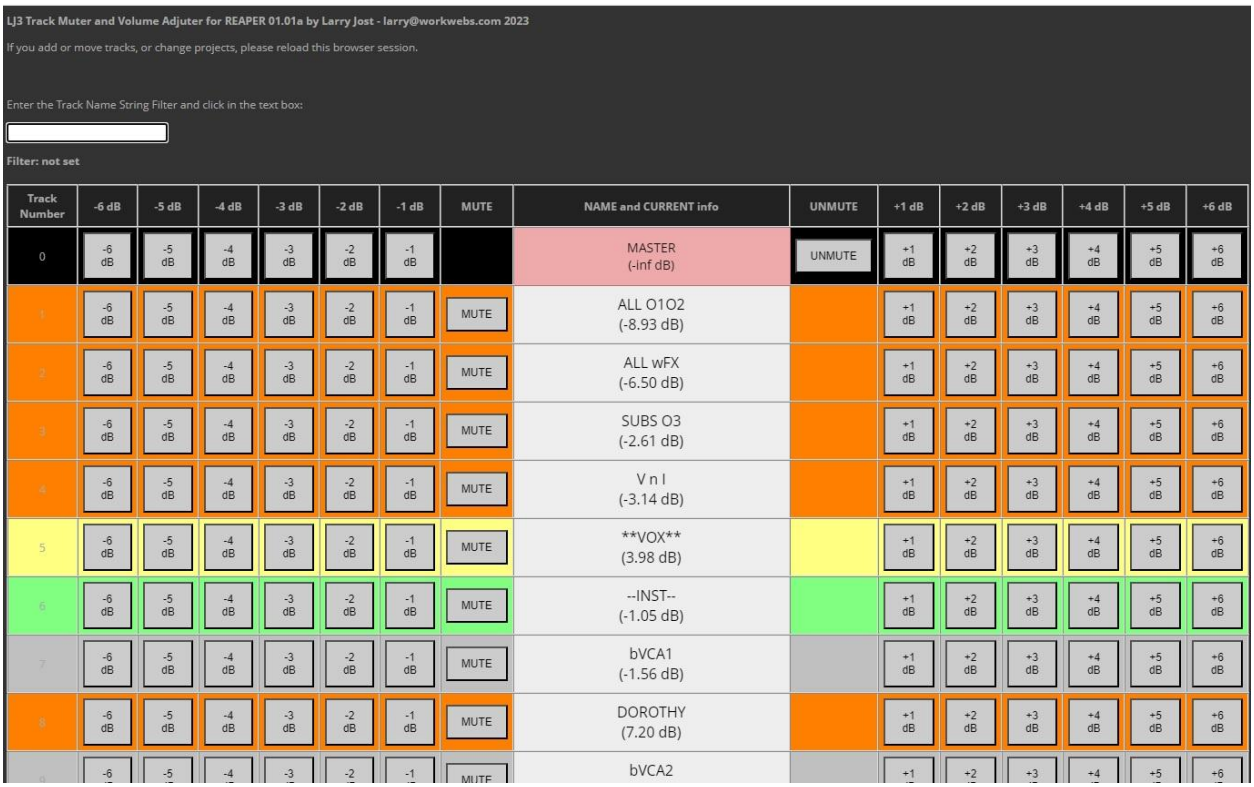


Figure 14 Track Muter and Volume Adjuster top section, the bottom section is the same as the Track Muter described earlier and not shown in this figure

Usage

This tool operates just like the track muter, and in fact includes the Track Muter as a lower section of the web page. However, the top section, shown in the figure, includes a separate track name filter that is just for the top portion of the page specific to the screen area that includes both mute and volume adjustment capabilities. After updating the filter text, right click in the filter area to make the change effective. Note that Folders will also appear here with an indication that they are folders above the track name to assist usage.

Installation

Please follow the same instructions as above for installing the Track Muter but use the script that includes the volume adjuster.

Technical Notes

The same technical notes as in the Track Muter, above, apply here. REAPER Fader level and adjustment is as indicated in the web page displayed. They are updated at the coded refresh interval just like the Track Muter.

Cue System

Overview

The cue system encompasses Windows Command Prompt Scripts and Cue files. For REAPER and for X32 it uses a batch program from Paul Vannatto named LT_Command that is used to send the Cues to the target device (see the links at the end of this document). Since LT_Command would not work with the WING (throwing an error upon receipt of a WING OSC response), the WING Cues are sent via script via Patrick-Gilles Maillot's WOSC tool (changes may be able to be made to allow all of these…x32, REAPER,Wing… to work via the yoggy sendosc in GitHub, but I have not tried that yet).

The Command Prompt scripts were developed to allow "easy" cue creation and execution. There are 2 parts to the scripts and the 2 parts are used in two phases of working with the cues.

You write your cues using text documents. The cue file you write is processed through a BUILD script that generates output files from each input cue file you wrote. The output files contain OSC commands. Your input files contain commands that the BUILD script understands and uses to create the output files.

Once the output files are created, a RUN script processes them in a sequence you define or allows you to enter a cue name to process a cue whenever you want. To "process" the output cue, the RUN script sends the script through the LT_Command program to send the OSC to the desired device, based on its IP address and port number.

> IMPORTANT: For REAPER to use the connection from LT_Command you must configure an interface in REAPER. To do this, go to Preferences->Control/OSC/web and create an OSC control surface control with pattern "default" and Mode Local port [receive only].

Remote control of the cue execution can be achieved with Bitfocus Companion with Vicreo Listener, a configuration is provided with this package.

There are BUILD scripts developed and available for creating OSC output files specific to the OSC required by REAPER or by the X32/M32 Behringer/Midas boards/racks or the Behringer WING.

The input and matching output cue files are named with a prefix that is specific to the target device for that cue. The extension of the input and output files ties the file to a specific project.

The RUN script can process cue output files for a project, and the cue files are destined to one or more devices of an available type (REAPER or x/m32 or WING).

You are also able to skip the build process and just make your own output files and they will still work with the RUN script and delivered per the IP and Port associated with the prefix letter used in the file name. This allows you to write OSC and send it to any device you want, either in a sequence or by name as desired.

ALL CUES, BUILD, and RUN scripts and the LT_COMMAND or wosc executable and associated installation files should all be placed the SAME DIRECTORY for this system to operate.

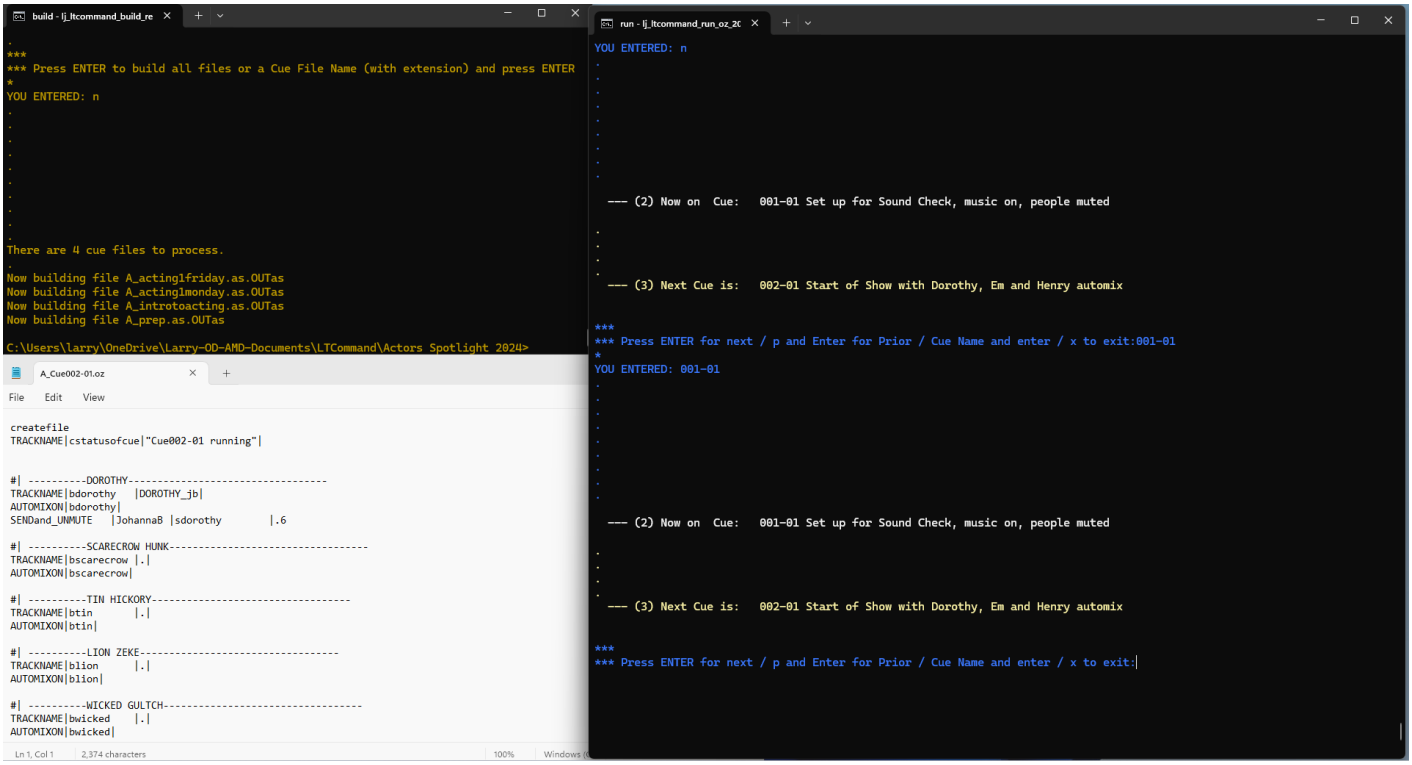


Figure 15 Top Left: Cue System BUILD script prompts sample. Top Right: Sample prompts form the Cue RUN script. Bottom Left: a sample portion of a Cue BUILD Input file.

Detail Using an Example for REAPER

An example will really help in understanding this, so an example is provided here. The syntax of the commands will be shown in detail later in this document.

The example here uses the input file content in the syntax for REAPER.

Using an editor (Notepad for example) you will edit the template BUILD script to define things for your usage.

The template BUILD script contains comments to help you in making these updates.

Build Script Edits

Type and Delay

The first lines to customize are the prefix letter that is associated with the target device and the setting of a delay time between commands being sent, in milliseconds.

In this example, we will prefix our Cue Input files with "A". Later in the RUN script we will set and IP and Port to associate with the "A" prefixed files. (This allows a change in IP to only require 1 update to the RUN script and all cues will go to the right IP because they are all tied together by the prefix letter.) Note that the value assigned to the Type reference (REAPER I the below) is for your reference only and it is not used in the script. It is up to you to use the correct BUILD script that matches your target environment.

Sample code in the BUILD file:

```
set Type.A=REAPER
set Delay.A=0
```

File Extension

This line in the BUILD script file also needs updating to reflect the project name. The BUILD and RUN scripts will limit themselves to the file extension you specify here and in the RUN script.:

```
set myFileExtension=oz
```

Track (Channel) Naming

Next, you need to let the system know the names you will use in your INPUT cues for each REAPER track you will reference. The Template provided assumes the custom REAPER layout will be used for the mixer I have configured. You can use the Cue system with any REAPER configuration you want though.

In the template I define the start of each section of tracks by a variable name so I can set up names relative to that offset. You do not need to do this. You can just use track numbers directly if you desire. However, every track you want to process commands against needs one or more names that you will use in your scripts.

For simplicity, I will provide an example that is not part of the template but is just illustrative. Let's say that the first track in REAPER is going to be used by an actor named Joe and Joe is playing the role of TheMan. In my scripts I would like to refer to Track 1 by 3 names: track1, joe, and theman.

In the BUILD script I would add these lines in the definition area:

```
set /a myChan.track1=1
set /a myChan.joe=1
set /a myChan.theman=1
```

Now, anywhere I use a reference to "track1" or "joe" or "theman" in the context of a channel or track, in an input cue I write the BUILD script will replace these names with a reference to Track 1. By using names in the cues, if they are moved to other tracks you only have to change the BULD file and rebuild the output cues. The cues do not need editing. These Names must not contain characters that a command script will misinterpret, for safety stick to letters and numbers. Other characters may be ok, but I have not verified which are and are not valid.

Let's say that this is the only track I want cues to operate against, so I am done defining reference names.

Send Identifiers (tracks to bus tracks sending, monitor sending, etc. based on your configuration)

In your cues, you will also be able to send tracks to specific sends. In this example, certain people have consistent sends. If you do not have dynamic sending and just have static routing you will not need to modify, you do not need to use any send identifiers.

For example, for a choir concert each person could be assigned to a SEND for a Soprano, Alto, Tenor or Bass group or a Solo track. In that scenario these 5 options are the only options a person has.

In setting up the tracks that will use these particular send identifiers, each track needs the same numbers of and positions of sends so that the offset in each does the same thing (send 1 is always to the Soprano Group, send 2 is always to the Alto group, etc. for every track for which these identifiers will be used). You can use different send identifiers for tracks that have a different set of sends configured in the system. If this is an x32, there are 16 sends for every track, those are your 16 buses. In REAPER this can be anything you configure.

In the SENDS Identifier designations note the following meaning of the SEND identifier content:
1 sets the send in this position to the volume level specified (0.0 through 1.0)
0 sets the send in this position to 0
X leaves the send alone
* terminates the processing of the send identifier
- is ignored and does not even count as a position in the send identifier, it is just used to make the strings easier to read.

For example:

```
set    mySend.soprano=1000-0*
set         mySend.alto=0100-0*
set       mySend.tenor=0010-0*
set         mySend.bass=0001-0*
set         mySend.solo=0000-1*
set     mySend.muteall=0000-0*
set  mySend.unchanged=*
```

The names specified here "solo","soprano", etc.) are used in the cue files in various commands.

For example an input cue file you write could contain:

SENDand_UNMUTE |joe |soprano |.6

This would, using this example, for the track "joe" references set the first send to volume level 0.6, the 2nd send for that track to volume 0, same for the 3rd, 4th, and 5th sends. It would then Unmute the "joe" track.

To avoid confusion, I recommend starting all send identifier names with an "s" to help ensure you know what is going on contextually.

Plugin Adjust identifiers (REAPER only, experimental)

In your cues, you will also be able to adjust an fx parameter using the same type of identifier as used for Sends, but with a different naming convention.

You will have to edit the BUILD script code to set the fx Parameter number you want to adjust this way. You can also duplicate the code or write a different command if you would like based on the example code. I do not yet have a use for this, so consider it experimental.

In the Plugin Identifier designations note the following meaning of the identifier content:
{anything else} runs whatever code you placed in the processing portion of the BUILD script for a value of whatever single character you placed in this position (you need to edit the code to have this do what you want)
x does nothing
* terminates the processing of the plugin adjust identifier
- is ignored and does not even count as a position , it is just used to make the strings easier to read.

For example:
set myPlugin.dothis=abc*
This will execute whatever you coded to happen when an "a" appears to the first plugin and will also do whatever you coded when a "b" appears to the second plugin of the specified track, and whatever you coded when a "c" appears to the 3rd plugin.

The default code only looks for "a". The command format is PLUGIN_ADJ|track|pluginadjidentifier| but you could add 2 more values if coded for.

When an "a" appears in position "n", in the sample code included in the script, the code will write the OSC to the output file to set the value of the 1st parameter of the nth plugin of the specified track to 1.

To avoid confusion, I recommend starting all send identifier names with a "p" to help ensure you know what is going on contextually.

Sample of Cue File Content (REAPER syntax) for reference

I can now write some cues. Let's say I want to mute Track 1 and Arm Track 1 and Name Track 1 to Joe. In Notepad, create a file named A_Setup.oz Recall that the first letter "A" will be tied to the IP and Port to send the Cue file to and the file extension oz indicates that this is part of project oz. A project can contain one or more file prefixes since a project may require sending OSC commands to one or more devices.

In the cue file place these "commands:"
createfile
MUTE|joe|
TRACKNAME|joe|Joe|
ARMON|joe|

Let's add another cue file to unmute this track. In Notepad create a file named A_Start.oz with this content:
createfile
UNMUTE||joe|

Great, we now have 2 cue files. The "createfile" command tells the RUN program to start a new Cue file of the proper name. Without this the new content written would be appended to the prior file by the same name. That is generally not a desired way to build cue files.

RUN Script Edits

Next, let's edit the template RUN script in Notepad.

IP, Port and File Extension

We need to set the IP, Port, and file extension. All files of the specified extension will be accessible by this copy of the RUN script. For just 1 destination and prefix letter of Cues to use, only 1 IP and Port combo is needed, with the one prefix letter used. If you want to control 2 REAPERS and 1 X32 you would need cues with 3 different prefix letters in their file names, all with the same extension. Each prefix letter will need definitions in the RUN script file.

```
set IP.A=192.168.0.107
set Port.A=10030
set myFileExtension=oz.OUToz
```

Cue List

Then each cue needs to be defined with a sequence number and some processing information. For this example:

```
set /a cuecount=0

 FOR EACH CUE

set /a cuecount+=1
```

```
set cue[%cuecount%].cuename=getready
set cue[%cuecount%].file=A_Setup
set cue[%cuecount%].cuedescr=get things ready
set cue[%cuecount%].afterpause=
set cue[%cuecount%].follow=
set cue[%cuecount%].execute=call

set /a cuecount+=1
set cue[%cuecount%].cuename=andgo
set cue[%cuecount%].file=A_Start
set cue[%cuecount%].cuedescr=We hear Joe now!
set cue[%cuecount%].afterpause=
set cue[%cuecount%].follow=
set cue[%cuecount%].execute=call
```

Using the Cues with the RUN script

Let's go through that a little. Cues are created with an ordered sequence. You do not need to process them in that sequence though if you do not want to. You can simply enter the "cuename" (getready or go in this example) to run the desired cue. To run them in sequence, you would just hit enter to go to the next cue. (You can back-up by entering "p".)

You can use a Cue File in more than one cue, but each cue needs a unique cuename value.

The cuedescr value is displayed in the command prompt window when the cue is run. You can wait a while before running a cue. The number of seconds to wait is set in the afterpause variable for the cue. If it is omitted then 0 is used.

If you provide a cue name assignment to the follow variable then the specified cue will run after the current cue is processed without you taking any further action. This is handy if you want to have a cue impact multiple devices because you can trigger one cue and the additional device cues will run automatically. Remember, an individual cue file is destined to a single device by a cue based on the prefix letter.

The last thing in each cue definition is an indication of how to process the cue. The choices are "call" or "start". If you specify "call" then the RUN script will process the cue and wait until it completes before allowing further action. If you specify "start" then an additional command prompt window is opened and runs the cue and the RUN script is ready for additional processing as soon as the cue is started.

There are lots of design choices to make when writing cues. Keep in mind that a lot of commands are available and can result in many OSC commands. If you anticipate running cues out of sequence, then each cue needs to get the entire system in a suitable state. If this involves setting 30 send values on each of 30 tracks, 30 track mutes, 30 track namings, and other things the cue may take longer than desired to execute.

In shows I have done I have limited the number of sends each track can be sent to so that I only need to set send levels on a subset of all sends involved. I also only set send levels when I unmute a track, letting the track being muted suffice to stop it from sending to monitors or other buses. I set up buses for some leads that only that lead uses. I set group buses anyone can be sent to. I set up several Lines

buses that non-leads will use. I stash all muted user tracks at the end of the cue to be sure to unmute needed tracks first. I limit how many tracks need renaming by naming buses appropriately but leaving the 48 main tracks to static names (adding initials to buses if/as appropriate). All of this helps reduce command volume or at least prioritizes what is most important.

After cues are built, the BUILD script is set up and the RUN script is set up, it is time to run the BUILD script. Open a command prompt window and change to the directory of the script and cues. The run your customized BUILD script. It will build an output cue file for each input cue file associated with project/file extension. If there are errors you will be notified. Once this completes, either open another command prompt window so you can get back to build quickly, or use the same window and execute the customized RUN script. Then follow the instructions to run your cues.

The full set of available input cue commands are described later in this document and are provided for REAPER or for X32/M32 or for the Behringer WING. The commands available to each device type differ so your cues need to use the appropriate BUILD file and cue content. BUILD templates are provided for REAPER and X32/M32 and the WING. Only 1 RUN script is required to process any of them though.

Note that you can run multiple RUN scripts on one or more networked computers. You may want to do this to aid in operational effectiveness or you may want to run sequential cues plus have non-sequential "commands" as cues in another setting. This lets you use utility cues at any time while not interrupting sequential flows. Using AutoHotKey (or similarly Bitfocus Companion) you can set up keyboard keys as single press commands for any of the RUN script. This is a great way to build a custom keyboard to process custom commands that are using OSC control!

CUES for REAPER

Each Cue file that you build contains commands that the BUILD script will read and create an expanded cue file for. Those cue files will be used by the RUN script later to run your cues.

Next is the detail on the commands that you place in a CUE file. Note that if you build these files in sequence that you limit the amount of adjustment you need to make in them and it really is pretty fast.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can used Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

```
reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

For the Current User and setting the sampleext extension to be recognized as text:

```
reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text
```

Note that Notepad++ is freely available and can be used to make mass changes to multiple files at once if needed to adjust your cue file.

Commands in the cue files are executed in the order received, so be sure you consider that when placing the content. Things that must be done quickly should be first.

Because cues can be run out of sequence, the policy leveraged for this example is to set everything in a track that can change in any cue in every cue. When things have to be done extremely quickly and back to back, that policy can have an exception. However, in my usage I name those cues with a "Q" in the name of the cue to identify it visually as a Quick cue and not a starting point.

Note that cue names and cue sequence are not the same. The sequence of cue file execution is specified in the RUN script.

Next below are the definitions of the commands that can be placed in the cue files. This will help you see what you can designate.

To build the Cue files for the RUN script to process, just run the BUILD script from within the directory with the other items and it will build all the cue expanded files that the RUN script will process.

One Example Cue File for reference

This is a cue from a run of Wizard of Oz, Jr. for an example to see. Other sample cues are provided with your download of this solution.

First, I run "createfile" to cause the BUILD script to create a new file instead of append the content to any existing output file that may already exist with this same name. The CUSTOMStartCue command could be used instead and it has added benefits, like creating a marker I the timeline at the time at which the cue was executed in the transport in REAPER. However, I did not want this to take place in this example so did not use that command.

I have a REAPPER track designated for cue information. That track has a defined script reference name of cstatusofcue. I set that to a string of text at the start and the end of the cue. In the provided track REAPER mixer layout, this appears in a text area of the screen to keep it available while mixing. In REAPER it is placed as the track name.

Then I have a comment line that starts with #|

The Dorothy character never moves from the bdorothy track, which is a bus track in REAPER. However, I set the bus name to "." when Dorothy is muted to avoid confusion (I mix at the bus level in the layout). I include the actual person's initials in the bus name so I can quickly find the track associated with the input for this character.

I then turn on AUTOMIX for this bus (this requires an automix plugin to be present on the bus. This is specific to use of a specific plugin for this purpose.

Next for the JohannaB track (who plays Dorothy) I set the send specified positionally by the sdorothy send identifier to volume 0.6 (which is 0 db in my scale) and unmute the track. Dorothy is then ready to be heard!

I have similarly dedicated buses for the Scarecrow/Hunk, Tin Man/Hickory, Lion/Zeke, Wicked Witch/Gulch. In this cue they are all muted. I mute them at the end of the script so that my unmutes take priority. Every Cue follows this same layout to make maintenance easy.

Next I can assign one or more source tracks to any of 5 line buses. In this example, Henry is unmuted for the Lines2 bus that is dynamically named HENRY_bg for this cue. The BeatriceG source track is routed to the blines2 bus.

I then place everyone who is muted into the "stash" area at the bottom of the cue.

You may find it add that I have a volume level and send string there. Once you see the options available for commands, you may ask why I didn't just use the MUTE command. The reason is that the SEND_ezMUTE command ignores everything in the command except the track name to mute. It does just mute the track. However, this command lets me keep that ignored content in place and I can just cut and paste it into position for whatever I am going to use it for. Then I just select "ezMUTE" with a double-click and paste or type in over it "UNMUTE" and set the send identifier and I am ready to go and have an organized easy to maintain layout consistently between cue files.

At the end of the cue file I turn the orchestra track on (in this case it is a stereo track from a recording) and rename the track so I can know what is happening from my mixing screen.

To start the next cue, just start with this one and save it under a different name and make adjustments.

```
createfile
TRACKNAME|cstatusofcue|"Cue095-01 running"|
#| ----------DOROTHY---------------------------------
TRACKNAME|bdorothy   |DOROTHY_jb|
AUTOMIXON|bdorothy|
SENDand_UNMUTE   |JohannaB    |sdorothy  |.6

#| ----------SCARECROW HUNK---------------------------------
TRACKNAME|bscarecrow |.|
AUTOMIXON|bscarecrow|

#| ----------TIN HICKORY---------------------------------
TRACKNAME|btin      |.|
AUTOMIXON|btin|

#| ----------LION ZEKE---------------------------------
TRACKNAME|blion     |.|
AUTOMIXON|blion|

#| ----------WICKED GULTCH---------------------------------
TRACKNAME|bwicked   |.|
AUTOMIXON|bwicked|

#| -------------------------------------------------
#| ----------LINE1---------------------------------
TRACKNAME|blines1|EM_zc|
AUTOMIXON|blines1|

#| ----------LINE2---------------------------------
TRACKNAME|blines2|HENRY_bg|
```

```
AUTOMIXON|blines2|
SENDand_UNMUTE  |BeatriceG  |slines2  |.6

#| ----------LINE3----------------------------------
TRACKNAME|blines3|.|
AUTOMIXON|blines3|

#| ----------LINE4----------------------------------
TRACKNAME|blines4|.|
AUTOMIXON|blines4|

#| ----------LINE5----------------------------------
TRACKNAME|blines5|.|
AUTOMIXON|blines5|

#| ----------GROUP1---------------------------------
TRACKNAME|bgroup1|-|
AUTOMIXON|bgroup1|

#| ----------GROUP2---------------------------------
TRACKNAME|bgroup2|-|
AUTOMIXON|bgroup2|

#| ----------STASHED---------------------------------
SENDand_ezMUTE  |AvaE     |slion    |.6
SENDand_ezMUTE  |ZoeyC    |slines1  |.6
SENDand_ezMUTE  |AlexandriaE|stin   |.6
SENDand_ezMUTE  |ShayleeW  |sscarecrow|.6
SENDand_ezMUTE  |SamiM    |slines3  |.6
SENDand_ezMUTE  |LeightonP |slines1  |.6
SENDand_ezMUTE  |FelixH   |slines1  |.6
SENDand_ezMUTE  |KatieB   |swicked  |.6
SENDand_ezMUTE  |JalonK   |slines2  |.6
SENDand_ezMUTE  |DevynnY  |slines1  |.6
SENDand_ezMUTE  |OliviaB  |sgroup1  |.6
SENDand_ezMUTE  |ElianaD  |sgroup1  |.6
SENDand_ezMUTE  |AlisonK  |sgroup1  |.6
SENDand_ezMUTE  |ChloeH   |sgroup1  |.6
SENDand_ezMUTE  |AnabellaS |sgroup1  |.6
SENDand_ezMUTE  |BriaR    |sgroup1  |.6
SENDand_ezMUTE  |LiliaC   |sgroup1  |.6

#| ----------ORCHESTRA---------------------------------
TRACKNAME|borch|ORCH|
AUTOMIXOFF|borch|
SENDand_UNMUTE |Orch     |sorch     |.6

TRACKNAME|cstatusofcue|"Cue095-01 completed-advance after Henry says OF COURSE WE BELIEVE YOU
DOROTHY"
```

REAPER Cue Content-Command Syntax and Definitions

Below is information on the commands that can be placed in the cue files you write. The BUILD script recognizes these and uses them to create similarly named output files that the RUN script will execute when you use the RUN script.

- NOTEs: Values designated as "myXxx value" are defined in the BUILD script as described earlier in this document.
- Commands are case-sensitive.
- Text values being passed must be in double quotes if they contain spaces.
- "Channel" and "Track" are used to mean the same thing.

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to an existing file by the same name if it exists. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.
Format:
createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:
1) it open a new output cue file based on the name of the input cue file
2) it renames the requested track to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
3) it sets the Delay value based on the value specified in the script.
Format:
CUSTOMStartCue|myChan value|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:
1) it renames the requested track to the specified text (perhaps the Cue name) to indicate to you that this is completed
2) it sets up a REAPER action call to 40157 to create a marker on the transport timeline
3) It names that marker by the text specified
Format:
CUSTOMEndCue| myChan value |text|

SENDand_UNMUTE

SENDand_UNMUTE is used to update SENDS to a volume specified by number and then UNMUTE the channel.
Format:
SENDand_UNMUTE|myChan value|mySend value |number |

SENDand_MUTE

SENDand_MUTE is used to mute the channel and then update SENDS to a volume specified by the number
Format:
SENDand_MUTE|myChan value|mySend value|number|

SENDand_ezMUTE

SENDand_ezMUTE is used to mute the channel. The remaining fields are ignored. This command exists so that you can copy and paste commands in cue files and undertake this simple mute without having to change the format of the command. For example, if you are using SENDand_UNMUTE to set sends and then unmute a track, you can just double click on UNMUTE and replace the text with ezMUTE. The track can be unmuted again by just going back and not having to retype the rest of the command.
Format:
SENDand_ezMUTE|myChan value|

SENDand_UNMUTE_NAME

SENDand_UNMUTE_NAME is used to update SENDS and then UNMUTE the channel and set the channel's display to text and volume level to number.
Format:
SENDand_UNMUTE_NAME|myChan value|mySend value|text|number|

SENDand_MUTE_NAME

SENDand_MUTE_NAME is used to mute the channel and then update SENDS to volume in number and set the channel's display name to text.
Format:
SENDand_MUTE_NAME|myChan value|mySend value|text|number|

TRACKPAN

TRACKPAN is used to pan the track. 0 is full left, 1 if full right, .5 is center.
Format:
TRACKPAN|myChan value|pan|

SELECT

SELECT is used to select a track. The Name is optional and if specified renames the track as well as selects it.
Format:
SELECT|myChan value|Name|

UNSELECT

UNSELECT is used to un-select a track. The Name is optional and if specified renames the track as well as unselects it.
Format:
UNSELECT|myChan value|Name|

UNMUTE

UNMUTE is used to UNMUTE the channel and set the channel's display name to text.
Format:
UNMUTE|myChan value|text|

MUTE

MUTE is used to MUTE the channel and set the channel's display name to text.
Format:
MUTE|myChan value|text|

MARKERName

MARKERName to update the name of the specified marker.
Format:
MARKERName|||ID|Name|

LASTMARKERName

LASTMARKERName to update the name of the last marker.
Format:
LASTMARKERName||Name|

GOTOMarker

GOTOMarker is used to go to the nth marker (not Marker number "N" but positionally Nth)
Format:
GOTMarker||number|

RECORD

RECORD is used to toggle start of transport.
Format:
RECORD

PLAY

PLAY is used to toggle play of transport.
Format:
PLAY

STOP

STOP is used to toggle stop of transport.
STOP

PAUSE

PAUSE is used to toggle pause of transport
Format:
PAUSE

FXBYPASS

FXBYPASS is used to bypass an FX for a track
Format:
FXBYPASS|my Chan value|fx number|

FXACTIVE

FXACTIVE is used to activate an FX for a track
Format:
FXACTIVE|my Chan value|fx number|

TRACKNAME

TRACKNAME is used to change the display name of a tracak
Format:
TRACKNAME|my Chan's Name|new display name|

TRACKVOLUME

TRACKVOLUME is used to change the volume of a track to number
Format:
TRACKVOLUME|my Chan value|number|

ACTIONi

ACTIONi takes an action number as an argument and sends it
A handy action number to keep in mind is 41743 which sends a refresh to subscribed interfacing systems. This does nothing for these scripts since they are listen only, but will be useful to refresh other devices such as TOUCHOSC interfaces. An action list can be found at https://forum.cockos.com/showthread.php?t=23236
Format:
ACTIONi||action number|

ACTIONs

ACTIONs takes an action text string as an argument and sends it
An action list can be found at https://forum.cockos.com/showthread.php?t=23236
Format:
ACTIONs||action number|

ARMON

ARMON is used to ARM the channel and set the channel's display name
ARMON|myChan's Name|Channel Display Name|

ARMOFF

ARMOFF is used to UN-ARM the channel and set the channel's display name
ARMOFF|myChan's Name|Channel Display Name|

AUTOMIXON

AUTOMIXON is a special case command. It is written to work with WT Automixer which allows up to 16 occurrences for auto mixing. What it actually does is sets fx n parameter 7 to 1 for the track specified. To work, WT Automix must be the nth fx in the track's chain. Edit the build code to change which fx is impacted.
AUTOMIXON|myChan's Name|

AUTOMIXOFF

AUTOMIXOFF is a special case command. It is written to work with WT Automixer which allows up to 16 occurrences for auto mixing. What it actually does is sets fx n parameter 7 to 0 for the track specified. To work, WT Automix must be the nth fx in the track's chain. Edit the build code to change which fx is impacted.

AUTOMIXON|myChan's Name|

PLUGIN_ADJ

PLUGIN_ADJ updates plugin settings based on a myPlugin pattern string and custom code (see details above).
PLUGIN_ADJ|myChan's Name|myPlugin's Name|optional Variable Value|optional Variable Value|

FXOPENUI

FXOPENUI is used to open an FX for a track
Format:
FXOPENUI|my Chan value|fx number|

FXCLOSEUI

FXCLOSEUI is used to close an FX for a track
Format:
FXCLOSEUI|my Chan value|fx number|

#|

This is the comment indicator
comments begin with #|
Format
#|

CUES for X32/M32

Each Cue file that you build contains commands that the BUILD script will read and create an expanded cue file for. Those cue files will be used by the RUN script later to run your cues.

Next is the detail on the commands that you place in a CUE file. Note that if you build these files in sequence that you limit the amount of adjustment you need to make in them and it really is pretty fast.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can used Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

For the Current User and setting the sampleext extension to be recognized as text:

reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

Note that Notepad++ is freely available and can be used to make mass changes to multiple files at once if needed to adjust your cue file.

Commands in the cue files are executed in the order received, so be sure you consider that when placing the content. Things that must be done quickly should be first.

Because cues can be run out of sequence, the policy leveraged for this example is to set everything in a track than can change in any cue in every cue. When things have to be done extremely quickly and back to back, that policy can have an exception. However, in my usage I name those cues with a "Q" in the name of the cue to identify it visually as a Quick cue and not a starting point.

Note that cue names and cue sequence are not the same. The sequence of cue file execution is specified in the RUN script.

Next below are the definitions of the commands that can be placed in the cue files. This will help you see what you can designate.

To build the Cue files for the RUN script to process, just run the BUILD script from within the directory with the other items and it will build all the cue expanded files that the RUN script will process.

Detail Using an Example for X32/M32

Channels

In the sample script you will see designations of channels like this:

set myChan.testchan1=01
set myChan.testchan2=02
set myChan.testchan3=03

Do not have spaces or characters that a CMD script will misinterpret in these names. Ideally stick to alphanumeric and underscore for safety.

For this example, "testchan1" is the name to identify the 1st channel in some X32 instance being controlled, etc. These are names you give to these channels for scripting. Note that the values MUST be 2 digits so include the leading 0 for Channels 0-9.

Numbers can be repeated, but names must not. Names are used across all controlled systems and each channel in those systems must be unique. It is these names that you will specify in your cues. This allows you to change things around in your system and only requires you to change the number here instead of making you edit each cue file if you move things around.

BUS IDENTIFIERS

In your cues, you will also be able to send tracks to specific BUSs.

The BUILD script is where you place your definitions, such as:

```
set  myBus.testbus1=01
set  myBus.testbus2=02
set  myBus.testbus3=03
set  myBus.testbus4=04
set  myBus.testbus5=05
```

testbus1 is the name used in Cue files you write to refer to Bus 1, for example. This allows you to change bus assignments in the BUILD definitions instead of in each cue file if needed. You can refer to BUSs by these names in cues.

SEND IDENTIFIERS

In the SENDS designations note the following meaning of the SEND strings:

1 sets the send in this position to ON, or Unmuted

0 sets the send in this position to OFF, or Muted

X leaves the send alone

* terminates the processing of the send string

- is ignored and does not even count as a position

Sample from this script:

```
set mySend.solo=1000*
set mySend.soloplusverb=0001*
set mySend.muteall=0000-0000-0000-0000*
set mySend.unchanged=*
```

The names specified here "solo", etc.) are used in the cue files.

Each 1 or 0 specified what to do with the send at that sequence position.

For example solo is:

*1000**

Position 1 is a 1 so BUS 1 Send would be set ON for the specified channel when this SEND is used in a command.

The other 0's in position 2, 3,4 mean BUS 2, 3, 4 Sends for the referenced Channel will be set to OFF.

– is a spacer and does not have any meaning or cause the position to increment, it just helps make things readable in the code.

* Ends the string of entries

DCA Assign Identifiers

You name your DCAs for reference in CUES for assigning DCAs to a Channel.

For the example script you will see:

set myDCAassign.testassigndca1=1
set myDCAassign.testassigndca2=5

BE CAREFUL—the numbers are not the DCA number, but rather are the DCA reference value the X32 uses.

Values range from 0 to 255.

- DCA 1 = 1
- DCA 2 = 2
- DCA 3 = 4
- DCA 4 = 8
- DCA 5 = 16
- DCA 6 = 32
- DCA 7 = 64
- DCA 8 = 128

This allows you to specify multiple DCAs for a single reference by adding values together.

For example, if you set testassigndca2 as a name of value 5, as above, then testassigndca2 refers to a combination of DCAs 1 and 3 since this is the of the values for DCAs 1 and 3 (1+4=5).

Note that this is used in setting a channel to be included in one or more DCAs. It is not how DCA names, colors, or mutes are handled.

DCA Identifiers

You name your DCAs for reference in CUES for DCA color, mute handling, and naming.

For the example script you will see:

set myDCA.testdca1=1
set myDCA.testdca2=2

These are names for DCAs when handling DCA attributes and range in value from 1-8. NOTE These are not used to assign channels to DCAs. See above for that.

Route Out Identifiers and Route Source Out

The x32 allows definition of 208 output choices for USER OUT. Since User Out is how you may want to designate how audio inputs are mapped to REAPER, being able to change these will be handy. If you use a DN9630, for example, you can send User Out 1-48 to an AES50 to go to the DN9630 to go to REAPER, with AES50x 1 referred to at input 1 in REAPER, etc.

While you can have 168 different audio sources connected to the x32, and a choice of sending any of 208 items as outputs, you can only send 48 at a time to this interface, for example, and can use User Out to specify them. By keeping an x32 and REAPER aligned, you can change routing this way to get up to 48 of any audio sources the x32 offers to REAPER at any one time, but you have access to all of them as choices to select from.

Jumping ahead of ourselves a bit, cues can contain a ROUTE_OUT line that looks like this:

ROUTE_OUT|xxx|yyy|

Where xxx is a Route Out Identifier you set up and yyy is a Route Name. Route Names are predefined in the BUILD script for you (you can change them). For Example RouteSourceOut "Local1" is set to a value of 1. For the x32, the value of 1 is defined as local input 1 connected to the board.

If you define testroute1 to equal 01 then place that in CUE file like this:

ROUTE_OUT|testoutroute1|LOCAL1

You would run the CUE to have the x32 set User Out 01 to be what x32 sees as the input at Local 1.

If you then map User Out 1 to AES50B 1 and connect a DN9630 to AES50 B and run REAPER as your DAW/Mixer you would get whatever device is connected to Local 1 as input 1 in REAPER. In this setup you can have up to 48 inputs flowing at any 1 time, and my changing routing in a CUE you can change which inputs those are as needed, If you manage REAPER at the same time, you would mute the track for the no longer needed device, change the routing on the x32, and unmute the REAPER track for the newly assigned device, This can all be done with these scripts and the SEND script, and you can make the CUEs automatically follow each other from one trigger cue or can run them manually,

Back to the sample provide BUILD script, you will see these as examples:

set myRouteOut.testoutroute1=01
set myRouteOut.testoutroute2=02
set myRouteOut.testoutroute3=03
set myRouteOut.testoutroute4=04

set myRouteOut.testoutroute5=05

Note that these MUST BE 2 digit values, so include the leading 0. Values range from 01 to 48.

You will also see these Route Names the give the x32 designated route number a name that matches the connection used.

set myRouteSourceOut.OFF=0
set myRouteSourceOut.LOCAL1=1
set myRouteSourceOut.LOCAL2=2
set myRouteSourceOut.LOCAL3=3
set myRouteSourceOut.LOCAL4=4
set myRouteSourceOut.LOCAL5=5
set myRouteSourceOut.LOCAL6=6
set myRouteSourceOut.LOCAL7=7
set myRouteSourceOut.LOCAL8=8
set myRouteSourceOut.LOCAL9=9
set myRouteSourceOut.LOCAL10=10

You may find renaming things to be useful, as the provided detail is just examples.

For example:

set myRouteOut.REAPERInput_1=01
set myRouteOut.REAPERInput_2=02
set myRouteOut.REAPERInput_3=03
set myRouteOut.REAPERInput_4=04
set myRouteOut.REAPERInput_5=05
set myRouteOut.REAPERInput_6=06
and

set myRouteSourceOut.OFF=0
set myRouteSourceOut.CELLO1=1
set myRouteSourceOut.CELLO2=2
set myRouteSourceOut.MIC1=3
set myRouteSourceOut.MIC2=4
set myRouteSourceOut.MIC3=5

Then you can say in a CUE:

ROUTE_OUT|REAPERInput_1|CELLO1
ROUTE_OUT|REAPERInput_2|CELLO2
ROUTE_OUT|REAPERInput_3|MIC1
ROUTE_OUT|REAPERInput_4|MIC2
ROUTE_OUT|REAPERInput_5|MIC3

This will set User Out to the specified devices.

Later, if you are not using MIC 1, you could reassign REAPER input 3 to something else. This gives you the ability to change what is sent on the fly among all connected equipment.

Be sure to keep a record of the mapping to numbers in the x32 definitions:

- 0 is OFF
- 1-32 are Local 1-32
- 33-80 are AES50 A 1-48
- 81-128 are AES50 B 1-48
- 129-160 are Card In 1-32
- 161-166 are AUX in 1-6
- 167 is TalkBack Internal
- 168 is Talkback External
- 169-184 are OUIT 1-16
- 185-200 are P16 1-16
- 201-206 are AUX 1-6
- 207 is Main L
- 208 is Main R

Route In Identifiers and Route Source In

The x32 allows definition of 168 input choices for USER IN. This is how you can designate what the x32 sees as inputs at any one time. In the example setup for the ROUTE OUT information above, if the DN9630 is connected to AES50 B then REAPER outputs 1-48 would go to AES50 B 1-48 for access at the x32. You may want to have a backup plan to pull these directly from the connections in case of REAPER failure, in which case you could have a cue prepped to revamp things. You also just may want to change inputs for other reasons or use this for setup.

Be sure to keep a record of the mapping to numbers in the x32 definitions that can be used for user in routing:

- 0 is OFF
- 1-32 are Local 1-32
- 33-80 are AES50 A 1-48
- 81-128 are AES50 B 1-48
- 129-160 are Card In 1-32
- 161-166 are AUX in 1-6
- 167 is TalkBack Internal
- 168 is Talkback External

The sample script includes the following:

set myRouteIn.testinroute1=01
set myRouteIn.testinroute2=02
set myRouteIn.testinroute3=03
set myRouteIn.testinroute4=04
set myRouteIn.testinroute5=05

Note that 2 digit values are needed and they can range from 0-32.

The myRouteSourceIn values in the BUILD script are the name references for these commands.

Colors

Colors for x32 scribble strips are passed in OSC as numbers, but designated in CUEs as names. A default set of names is provided in the BUILD script, but you can change these.

These lines define the 15 color choices:

- *set myColor.OFF=0*
- *set myColor.RD=1*
- *set myColor.GN=2*
- *set myColor.YE=3*
- *set myColor.BL=4*
- *set myColor.MG=5*
- *set myColor.CY=6*
- *set myColor.WH=7*
- *set myColor.OFFi=8*
- *set myColorRDi=9*
- *set myColor.GNi=10*
- *set myColor.YEi=11*
- *set myColor.BLi=12*
- *set MyColor.MGi=13*
- *set myColor.CYi=14*
- *set myColor.WHi=15*

In CUEs you refer to these by OFF, RD, etc.

Card Commands

The xLive card can be manipulated in CUES. In the X32 the various actions are numbers, but they are mapped to names for CUE use.

The following are defined in the script, but you can change these

- *set myCard.STOP=0*
- *set myCard.PLAY=2*
- *set myCard.PAUSEPLAY=1*
- *set myCard.RECORD=3*

In CUEs you refer to these by STOP, PLAY, etc.

USB Stick Commands

The USB stick can be manipulated in CUES. In the X32 the various actions are numbers, but they are mapped to names for CUE use.

The following are defined in the script, but you can change these

- *set myTape.STOP=0*
- *set myTape.PLAY=2*
- *set myTape.PAUSEPLAY=1*

- *set myTape.RECORD=4*
- *set myTape.PAUSERECORD=3*

In CUEs you refer to these by STOP, PLAY, etc.

CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS X32

NOTE: Values designated as "myXxx value" are defined in the x32 BUILD script and described above.

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to the existing file. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.
Format:
createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:
1) it calls createfile to open a new output cue file based on the name of the input cue file
2) it renames the main stereo channel to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
3) it sets the Delay value
Format:
CUSTOMStartCue|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:
1) it renames Main Stereo Channel to the specified text (perhaps the Cue name) to indicate to you that this is completed
Format:
CUSTOMEndCue|text|

CHAN

CHAN is used to mute or unmute a channel
Format:
CHAN|myChan value|MUTE or UNMUTE

CHANNAME

CHANNAME is used to set a channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
CHANNAME| myChan value|text

CHANCOLOR

CHANCOLOR is used to set the color for a channel in the scribble strip. The Color Names are set in the BUILD script.
Format:
CHANCOLOR| myChan value|Color Name

CHANSEND

CHANSEND is used to set the enable or disable a send from a specified channel.
Format:
CHANSEND|myChan value| myBus value |MUTE or UNMUTE

CHANDCA

CHANDCA is used to set the a Channel into one or more DCAs.
Format:
CHANDCA| myChan value|myDCAassign value|

DCA

DCA is used to mute or unmute a DCA
Format:
DCA|myDCA value|MUTE or UNMUTE

DCANAME

DCANAME is used to set a DCA name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
DCANAME| myDCA value|text

DCACOLOR

DCACOLOR is used to set the color for a DCA in the scribble strip. The Color Names are set in the BUILD script.
Format:
DCACOLOR| myDCA value |myColor value

BUS

BUS is used to mute or unmute a BUS
Format:
BUS|myBus value|MUTE or UNMUTE

BUSNAME

BUSNAME is used to set a BUS name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
BUSNAME| myBus value|text

BUSCOLOR

BUSCOLOR is used to set the color for a BUS in the scribble strip.
Format:
BUSCOLOR|myBus value|myColor value

MAINST

MAINST is used to mute or unmute the Main Stereo Channel
Format:
MAINST|MUTE or UNMUTE

MAINSTNAME

MAINST is used to set a MAIN Stereo Channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
MAINSTNAME|text

MAINSTCOLOR

MAINSTCOLOR is used to set the color for the Main Stereo channel in the scribble strip.
Format:
MAINSTCOLOR|myColor value

MAINM

MAINM is used to mute or unmute the Main Mono Channel
Format:
MAINM|MUTE or UNMUTE

MAINMNAME

MAINM is used to set a MAIN Mono Channel name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
MAINMNAME|text

MAINMCOLOR

MAINMCOLOR is used to set the color for the Main Mono channel in the scribble strip.
Format:
MAINMCOLOR|myColor value

SENDand_UNMUTE

SENDand_UNMUTE is used to update BUS SENDS and then UNMUTE the channel.
Format:
SENDand_UNMUTE|myChan value|mySend value|

SENDand_MUTE

SENDand_MUTE is used to mute the channel and then update BUS SENDS
Format:

SENDand_MUTE| myChan value|mySend value|

ROUTE_OUT

ROUTE_OUT is used to define a source and target for USER OUT
Format:
ROUTE_OUT|myRouteOut value|myRouteSourceOut value

ROUTE_IN

ROUTE_IN is used to define a source and target for USER IN
Format:
ROUTE_IN| myRouteIn value|myRouteSourceIn value

USB_RECORDER

USB_RECORDER is used control play and record for the USB recorder
Format:
USB_RECORDER|myTape value|

LIVE_RECORDER

LIVE_RECORDER is used control play and record for the LIVE SD recorder
Format:
LIVE_RECORDER|myCard value|

SAVESCENE

SAVESCENE is used to save a scene in slot 0-99 and name it and set a scene note. Enclose strings with spaces in them in quotes.
Format:
SAVESCENE|mySceneNumber|Name|Note|

LOADSCENE

LOADSCENE is used to load a scene from slot 0-99.
Format:
LOADSCENE|mySceneNumber|

SAVESNIPPET

SAVESSNIPPET is used to save a snippet in slot 0-99 and name it. Enclose strings with spaces in them in quotes. Note that the filters used are those already assigned to this snippet slot. Therefore, you can use this to replace content of a snippet but not save a brand new snippet.
Format:
SAVESNIPPET|mySnippetNumber|Name

LOADSNIPPET

LOADSNIPPET is used to load a snippet from slot 0-99.
Format:
LOADSNIPPET|mySnippetNumber|

#|
This is the comment indicator
comments begin with #|
Format
#|

CUES for the Behringer WING

Each Cue file that you build contains commands that the BUILD script will read and create an expanded cue file for. Those cue files will be used by the RUN script later to run your cues.

Next is the detail on the commands that you place in a CUE file. Note that if you build these files in sequence that you limit the amount of adjustment you need to make in them and it really is pretty fast.

Also note that if you want to set your extension to be recognized by Windows as a text file so you can used Notepad with it that you can run this CMD:

For the Local Machine and setting the sampleext extension to be recognized as text:

reg add HKLM\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

For the Current User and setting the sampleext extension to be recognized as text:

reg add HKCU\SOFTWARE\Classes\.sampleext /v PerceivedType /t REG_SZ /d text

Note that Notepad++ is freely available and can be used to make mass changes to multiple files at once if needed to adjust your cue file.

Commands in the cue files are executed in the order received, so be sure you consider that when placing the content. Things that must be done quickly should be first.

Because cues can be run out of sequence, the policy leveraged for this example is to set everything in a track than can change in any cue in every cue. When things have to be done extremely quickly and back to back, that policy can have an exception. However, in my usage I name those cues with a "Q" in the name of the cue to identify it visually as a Quick cue and not a starting point.

Note that cue names and cue sequence are not the same. The sequence of cue file execution is specified in the RUN script.

Next below are the definitions of the commands that can be placed in the cue files. This will help you see what you can designate.

To build the Cue files for the RUN script to process, just run the BUILD script from within the directory with the other items and it will build all the cue expanded files that the RUN script will process.

Detail Using an Example for the Behringer WING

Channels

In the sample script you will see designations of channels like this:

set myTrackVar.iLCL1=1
set myTrackVar.iLCL1.type=io/in/LCL

set myTrackVar.iLCL2=2
set myTrackVar.iLCL2.type=io/in/LCL

set myTrackVar.iLCL3=3
set myTrackVar.iLCL3.type=io/in/LCL

Do not have spaces or characters that a CMD script will misinterpret in these names. Ideally stick to alphanumeric and underscore for safety.

Each TrackVar in the WING cue BUILD file has not only a number but also a type. The TYPE specifies the OSC string used to prefix the command that the CUE generates. In these examples, the "inputs" are given names, a value, and a type.

If in a script we reference to iLCL3, it has a value of 3 and a type of io/in/LCL. This lets us use iLCL3 to name or mute or whatever the LCL input number 3.

An example use in a CUE file would be:

TRACKCOLOR|iLCL3|DARKBLUE|
TRACKNAME|iLCL3|"LCL3"|
TRACKMODE|iLCL3|"M"|

This would set the color of the Local 3 input to DARKBLUE, would, would set its display name to LCL3 and would set it as a MONO input.

Type values can be: ch, aux, mtx, bus, dca, main for general use for channels, auxs, matrixes, busses, dcas and main track types.

Type can be of the form io/in/LCL (where LCL is an example) to impact an io/in or io/ou/LCL (where LCL is an example) or any other OSC desired.

Note that all types do not support all commands, see the manual of OSC commands.

SEND IDENTIFIERS

In the SENDS designations note the following meaning of the SEND strings:

1 sets the send in this position to ON, or Unmuted

0 sets the send in this position to OFF, or Muted

X leaves the send alone

* terminates the processing of the send string

- is ignored and does not even count as a position

Sample from a script:

:: SENDS NEVER CHANGE
*set mySend.sdonothing=**

*set mySend.svoxsetup=111-000-00-00-111111**
*set mySend.sinstsetup=000-111-11-00-111111**

The names specified here are used in the cue files.

Each 1 or 0 specified what to do with the send at that sequence position.

For example solo is:

*1000**

Position 1 is a 1 so BUS 1 Send would be set ON for the specified channel when this SEND is used in a command.

The other 0's in position 2, 3,4 mean BUS 2, 3, 4 Sends for the referenced Channel will be set to OFF.

– is a spacer and does not have any meaning or cause the position to increment, it just helps make things readable in the code.

* Ends the string of entries

Colors

Colors for WING scribble strips are passed in OSC as numbers, but designated in CUEs as names. A default set of names is provided in the BUILD script, but you can change these.

These lines define the color choices:

- *set myColor.LIGHTBLUE=1*
- *set myColor.MEDIUMBLUE=2*
- *set myColor.DARKBLUE=3*
- *set myColor.TURQUOISE=4*
- *set myColor.GREEN=5*
- *set myColor.OLIVEGREEN=6*
- *set myColor.YELLOW=7*
- *set myColor.ORANGE=8*
- *set myColor.RED=9*
- *set myColor.CORAL=10*
- *set myColor.PINK=11*
- *set myColor.MAUVE=12*
- *set myColor.GNi=10*
- *set myColor.YEi=11*
- *set myColor.BLi=12*
- *set MyColor.MGi=13*
- *set myColor.CYi=14*

- *set myColor.WHi=15*

In CUEs you refer to these by GREEN, YELLOW, etc.

Connection Commands

The WING channels need to have connections set in place to indicate the source for the channel. This is done by a Group designation and a number designation within the group. See the WING OSC manual for details, but for example, a Channel connection can be from any of these groups: OFF, LCL, AUX, A, B, C, SC, USB, CRD, MOD, PLAY, AES, USR, OSC, BUS, MAIN, MTX with a number from 1-64. Likewise for an AUX.

In the BUILD script you may say:

set myCONNVar.micSEN1.type=B
set myCONNVar.micSEN1.num=1

to indicate that a reference to micSEN1 refers to AES50 B 1.

In a CUE you could use:

TRACKGROUPIN|cJenny|micSEN1|

to indicate that the channel named cJenny is to be connected to whatever micSEN1 indicated, namely AES50 B 1.

TAG Commands

TAGs in the WING are used for a variety of uses, but in the CUEs they are set up specifically for DCA handling. This is done with the myTAGVar variables such as:

set myTAGVar.dca1="#D1"
set myTAGVar.dcaJO="#D1"

Multiple DCA tag can be used, just separate them with commas. DCA Tags are #D1…#D16. With the above in the build file, by cues can reference dca1 or dcaJo and either will refer to that #D1 with is the tag for the first DCA.

CUE FILE CONTENT COMMAND FORMATS AND DEFINITIONS WING

NOTE: Values designated as "myXxx value" are defined in the WING BUILD script and described above.

BE SURE to see the details and sample under SETLATESTDCA (Set Latest DCA) and SETTRACKTOLATESTDCA (Set Track to Latest DCA) cue command and sample below. This is a wonderful way to work your cues for running a show.

createfile

createfile causes the system to build a new output cue file. Without this, all added commands will append to the existing file. This must be used once at the start of a Cue file but it can also be implemented with the CUSTOMStartCue command which will also perform this function and more.
Format:
createfile|

CUSTOMStartCue

CUSTOMStartCue is used to start a cue using a custom set of commands as follows:
1) it calls createfile to open a new output cue file based on the name of the input cue file
2) it renames the matrix 8 channel to "*" followed by the specified text (perhaps the Cue name) to indicate to you that this is running
3) it sets the Delay value
Format:
CUSTOMStartCue|text|

CUSTOMEndCue

CUSTOMEndCue is used to end a cue using a custom set of commands as follows:
1) it renames Matrix 8 Channel to the specified text (perhaps the Cue name) to indicate to you that this is completed
Format:
CUSTOMEndCue|text|

TRACKMUTE

TRACKMUTE is used to mute or unmute a track
Format:
TRACKMUTE|myTrackVar's name|MUTE or UNMUTE|

TRACKLED

TRACKMLED is used to turn the track LED on or off
Format:
TRACKLED|myTrackVar's name|ON or OFF|

TRACKNAME

TRACKNAME is used to set a track name in the scribble strip to a text string (if there are spaces, enclose the string in quotes)
Format:
TRACKNAME| myTrackVar's name |text|

TRACKCOLOR

TRACKCOLOR is used to set the color for a track in the scribble strip. The Color Names are set in the BUILD script (sample: LIGHTBLUE, MEDIUMBLUE, DARKBLUE, TURQUOISE, GREEN, OLIVEGREEN, YELLOW, ORANGE, RED, CORAL, PINK, MAUVE).

Format:
TRACKCOLOR| myTrackVar's name |Color Name|

TRACKICON

TRACKICON is used to set a track icon in the scribble
Format:
TRACKICON| myTrackVar's name |icon number|

TRACKPAN

TRACKPAN is used to set a pan a track. Values are -100 to 100
Format:
TRACKPAN| myTrackVar's name |number|

TRACKPOSTINS

TRACKPOSTINS is used to assign a post insert for a track.
Format:
TRACKPOSTINS| myTrackVar's name |FX or AUTO_X or AUTO_Y|

TRACKPROC

TRACKPROC is used to assign a set the GEDI in the desired order for processing. See the OSC manual for options.
GEDI, GEID, GIED, IGED…
Format:
TRACKPROC| myTrackVar's name|text|

TRACKTAG

TRACKTAG is used to assign a TAG value to a track.
Format:
TRACKTAG| myTrackVar's name|myTagVar's name|

TRACKTAGMUTE

TRACKTAGMUTE is used to assign a TAG value to a track and mute or unmute the track
Format:
TRACKTAGMUTE| myTrackVar's name|myTagVar's name|MUTE or UNMUTE|

TRACKGROUPIN

TRACKGROUPIN is used to set a GROUP connection name and number. It is best to do this while the track is muted.
Format:
TRACKGROUPIN| myTrackVar's name|myCONNVar's name|

TRACKSEND

TRACKSEND is used to set the enable or disable a send from a specified track.
Format:
TRACKSEND| myTrackVar's name | mySend's name|MUTE or UNMUTE

SENDand_UNMUTE

SENDand_UNMUTE is used to update SENDS and then UNMUTE the track.
Format:
SENDand_UNMUTE| myTrackVar's name | mySend's name |

SENDand_MUTE

SENDand_MUTE is used to update SENDS and then MUTE the track.
Format:
SENDand_MUTE| myTrackVar's name | mySend's name |

USB_PLAY

USB_PLAY is used to control playing for the USB recorder
Format:
USB_PLAY|STOP or PAUSE or NEXT or PREV or PLAYFILE|

USB_PLAYFILE

USB_PLAYFILE is used to set the file name to play for when the PLAYFILE is sent with USB_PLAY
Format:
USB_PLAYFILE|full file name|

USB_RECORD

USB_RECORD is used to control recording for the USB recorder
Format:
USB_RECORD|STOP or REC or PAUSE or NEXT or NEWFILE|

LIVE_RECORDER

LIVE_RECORDER is used control the LIVE Card Recorder
Format:
LIVE_RECORDER|1 or 2|STOP or PLAY or PPAUSE or RECORD|

LOADSCENE

LOADSCENE is used to load a scene from its Tag Number
Format:
LOADSCENE|Scene Tag Number|

NAVSCENE

LOADSCENE is used to load a scene
Format:
NAVSCENE|GOPREV or GONEXT or GO or PREV or NEXT|

AUTOMIXXON
AUTOMIXXOFF
AUTOMIXYON
AUTOMIXYOFF
These are used to turn the specified AUTOMIX on or off.

Format:

AUTOMIXXON

Or

AUTOMIXXOFF

Or

AUTOMIXYON

Or

AUTOMXYOFF

CUSTOM_a_DCA_ONMUTED

Custom_a_DCA_ONMUTED is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to the specified text
2. It turns the Track's LED off
3. It mutes the track
4. In the current code is code to sets color YELLOW but this is presently commented out

Format:
Custom_a_DCA_ONMUTED|MyTrackVar's Name|text|

CUSTOM_a_DCA_ONUNMUTED

Custom_a_DCA_ONUNMUTED is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to the specified text
2. It turns the Track's LED on
3. It unmutes the track
4. In the current code is code to sets color YELLOW but this is presently commented out

Format:
Custom_a_DCA_ONUNMUTED|MyTrackVar's Name|text|

CUSTOM_a_DCA_OFF

Custom_a_DCA_OFF is a custom code area used to set up a DCA track (or other track)

1. It sets the Track's Name to "."
2. It turns the Track's LED off
3. It mutes the track
4. In the current code is code to sets color DARKBLUE but this is presently commented out

Since text is ignored you can just change the command from a text matching
the Custom_a_DCA_ONMUTED or Custom_a_DCA_ONUNMUTED commands
Format:
Custom_a_DCA_OFF|MyTrackVar's Name|text{but text is ignored}|

Custom_a_CHAN_ON

Custom_a_CHAN_ON is a custom code area used to set up a channel track possibly with AUTO MIX and with a DCA assignment

1. It sets the Track's Post Insert to the specified text (like FX or AUTO_X or AUTO_Y)
2. It turns the Track's Post Insert On
3. It sets the Track's LED on
4. It turns the Track's Post Insert On
5. It sets the Track's DCA tags to match the specified DCA Variable content
6. It runs the tracks through the SENDand_UNMUTE sequence based on the Send String content
7. It colors the track as indicated in COLOR
8. It names the track as indicated in TEXT
9. It unmutes the track

Format:
Custom_a_CHAN_ON|MyTrackVar's Name|MyDCAVar's Name|post insert text|MySendString name|COLOR|TEXT|

Custom_a_CHAN_OFF

Custom_a_CHAN_OFF is a custom code area used to set up a channel track possibly with an "off" status…meaning:

1. It sets the Track's Post Insert to FX
2. It turns the Track's Post Insert On
3. It sets the Track's LED off
4. It sets the Track's DCA tags to match the "dcapark" DCA Variable content
5. It runs the tracks through the SENDand_MUTE sequence for "spark" including that it mutes the track although LED is off, COLOR is set to dark blue
6. It names the track as indicated in TEXT

The inclusion of and non-use of parameters makes it easier to keep editing cues and toggling between these 2 custom commands.

Format:
Custom_a_CHAN_OFF|MyTrackVar's Name|MyDCAVar's Name{ignored}|post insert text {ignored}|MySendString name{ignored}|COLOR{ignored}|TEXT|

SETLATESTDCA

:SETLATESTDCA sets the latest DCA to the specified DCA

Format:
SETLATESTDCA|TagVar's name or xxx for not having one|myTrackVar for the DCA|Name to set on the DCA Track

SETTRACKTOLATESTDCA

SETTRACKTOLATESTDCA sets the specified Track to have the Tag from the SETLATESTDCA value

- It sets the tags according to the TagVar in SETLATESTDCA and
- If the Tag content is xxx then it mutes the track and turns off the LED (and removes all tags)
- otherwise, it does set the tags according to the TagVar in SETLATESTDCA and turns on the LED and Unmutes the specified track

Format:
SETTRACKTOLATESTDCA|myTrackVar's name|

These 2 commands allow you to just place a SETLATESTDCA command in a CUE and then follow it by the SETTRACKTOLATESTDCA commands for all tracks you want in that DCA.

If you use xxx in the SETTRACKTOLATESTDCA name then whatever tracks that follow will be muted.

With these you can cut and past SETTRACKTOLATESTDCA text lines under the desired DCAs to manage the cues. This is very efficient and easy to manage.

Here is sample CUE file to demonstrate this:

```
CUSTOMStartCue|"116 running"|
#| --------- overall auto mix - everyone is in X exc ENS-only ----------------------
AUTOMIXXON

#| ---------DCA ASSIGNMENTS----------------------
SETLATESTDCA      |dcaJo|dJo      |"JO"
SETTRACKTOLATESTDCA  |cJenny        | # Jenny    is Jo

SETLATESTDCA      |dcaAmy|dAmy     |"."

SETLATESTDCA      |dcaMeg|dMeg     |"."

SETLATESTDCA      |dcaBeth|dBeth    |"."

SETLATESTDCA      |dcalines1|dlines1|"MARMEE"
SETTRACKTOLATESTDCA  |cAlecia       | # Alecia   is Marmee

SETLATESTDCA      |dcalines2|dlines2|"BHAER"
SETTRACKTOLATESTDCA  |cWill         | # Will     is Bhaer

SETLATESTDCA      |dcalines3|dlines3|"."

SETLATESTDCA      |dcalines4|dlines4|"."
```

```
SETLATESTDCA      |dcalines5|dlines5|"."

SETLATESTDCA      |dcaens1|dens1   |"."

SETLATESTDCA      |dcaens2|dens2   |"."

SETLATESTDCA      |dcaens3|dens3   |"."

SETLATESTDCA      |xxx
SETTRACKTOLATESTDCA |cApril       | # April   is Aunt March
SETTRACKTOLATESTDCA |cJulia       | # Julia   is Meg
SETTRACKTOLATESTDCA |cPeter       | # Peter   is Laurie and Rodrigo
SETTRACKTOLATESTDCA |cJohn        | # John    is Laurence
SETTRACKTOLATESTDCA |cQuentin     | # Quentin  is Brooke and Braxton
SETTRACKTOLATESTDCA |cGrace       | # Grace   is Amy
SETTRACKTOLATESTDCA |cSarah       | # Sarah   is Beth
SETTRACKTOLATESTDCA |cJamie       | # Jamie   is Kirk
SETTRACKTOLATESTDCA |cSyr         | # Syr     is Knight
SETTRACKTOLATESTDCA |cTia         | # Tia     is Hag
SETTRACKTOLATESTDCA |cAllison     | # Allison  is Troll
SETTRACKTOLATESTDCA |cDylan       | # Dylan   is Rodrigo 2
SETTRACKTOLATESTDCA |cKristyn     | # Kristyn  is Ensemble
SETTRACKTOLATESTDCA |cKatelyn     | # Katelyn  is Ensemble
SETTRACKTOLATESTDCA |cSabrina     | # Sabrina  is Clarissa
SETTRACKTOLATESTDCA |cJenni       | # Jenni   is Ensemble

CUSTOMEndCue|"116 done"|
```

#|
This is the comment indicator
comments begin with #|
Format
#|

REAPER Mixer

Part of this ecosystem is a REAPER mixer configuration. The sample REAPER file is included with the package that is downloaded.

Specifications

- Uses Open Stage Control for the touch control mixer GUI
- Open Stage Control json file provided for the solution
- 48 inputs in the design as released, space for more
- Outputs constrained by interface capabilities
- 120 small faders (best for general use tracks, effects tracks)
- 64 large faders (can be used for tracks, busses, VCAs, etc.)
- 20 rotary faders (can be used for VCAs or monitor, for example)
- 21 button tracks (can be used for mute groups, for example)
- multiple "comment/message" track for script to use for sharing info
- PFLs for the 48 inputs
- AFLs for all other items that are in the audio path
- Routing and track usage is flexible as configured within REAPER

Overview of REAPER setup

There are tracks as indicated above present ready for you to name, route, and define as per your needs.

Input Tracks Sample

When I use them, the 48 input tracks have no plugins. They are always unmuted. They receive input from the hardware. This lets them be recorded in REAPER in their original input form and not be touched or moved. This also allows a Pre-Fade listener to be established. The Input Tracks are sent to the other Tracks and to the PFL tracks.

Channel Tracks

The small fader and large fader tracks are all stereo. There are 120 small faders and 64 large faders I the Open Stage Control setup and in the REAPER sample. They generally have a channel strip plugin applied, but there are no restrictions on what you can do within REAPER

Button Tracks

There are 21 Button Tracks, initially designed for Mute Groups. In REAPER's Track Group page, set the follows to the se tracks, which are the Leads. You can do a lot more than mute with these.

PFL and AFL Tracks

The Pre-fade Listen tracks receive the Input Tracks. There are 48 of them. The After Fade Listen tracks, receive content from all the other small and large fader tracks. These route to Output(s) from which they can be monitored. They can both go to the same output if you want. These tracks are kept muted except when being listened to. Just unmute and listen. The Track Muter GUI works well for this but the GUI also provides this capability for most of the tracks.

Rotary Tracks

There are 20 rotary tracks. They can receive anything you want, pre or post fader, as you configure them in REAPER. They could feed Outputs for audio use.

Comment/Message Track

This main comment message track is used in the GUI to display text as defined in cues. It of course can also be seen in REAPER's GUI. The Track Name is set in Cues and displayed for information in the GUI.\

GUI Overlays

At the start and end of the Open Stage Control definitions are overlays you can use to further obfuscate totally unused tracks and to give a color to groups of tracks. Just enter the Open Stage Control Editor and select and edit the background and overlay folder content to move things around or make your own or hide what is in the sample.

The GUI Layout

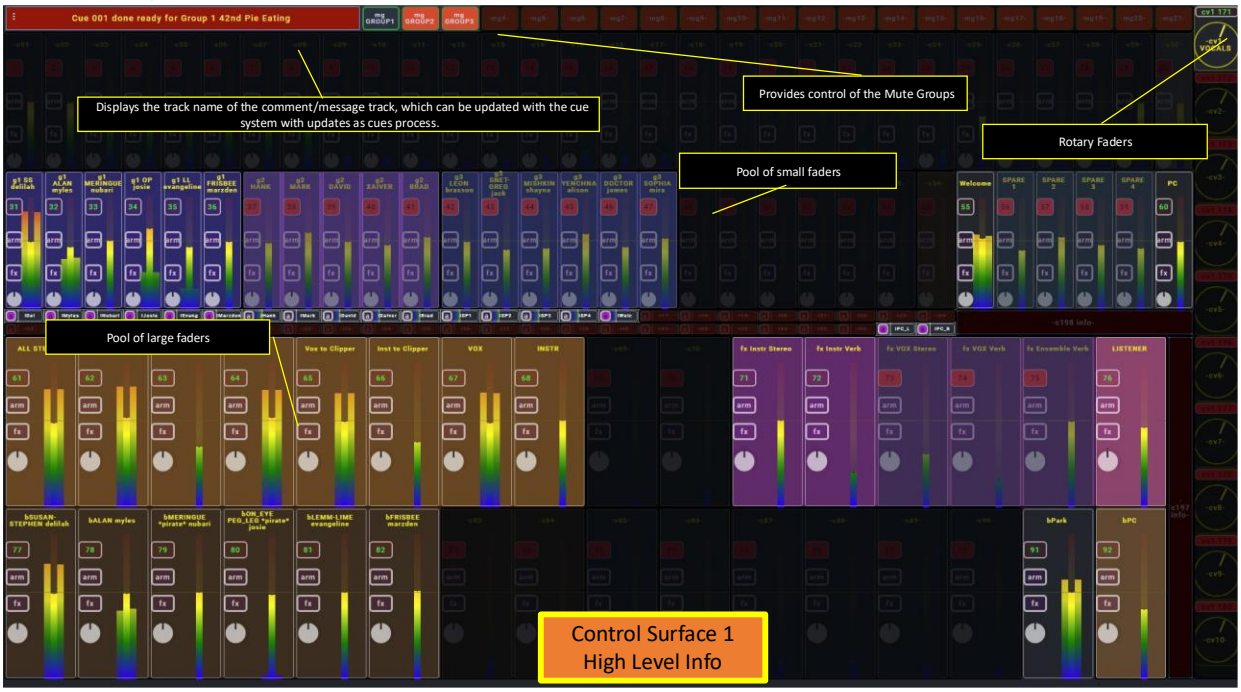


Figure 16 Primary GUI layout high level

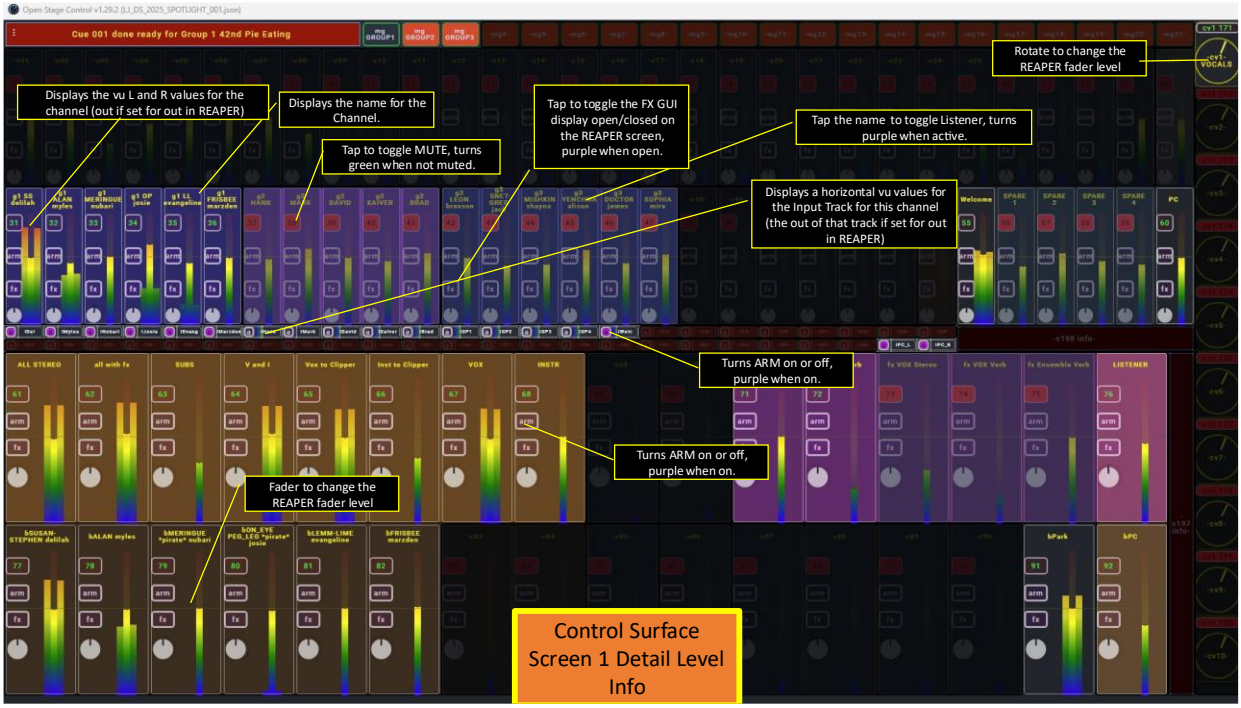


Figure 17 Primary GUI Layout detail

The GUI is designed for a full HD screen and works well on a monitor that is at least 27 inches diagonally measured. A touchscreen is recommended.

The faders interact with REAPER faders.

USAGE NOTE: I have used multiple touch screens. I have found that some require a heavy touch and some require a light touch. If a heavy or a light touch is not getting responsiveness, try changing your touch.

Setting up Tracks in REAPER

- Use the ROUTING matrix to set up routing.
- In REAPER Preferences->Audio->Mute/Solo set Do Not process muted tracks (even if FX UI is open). This saves resources.
- In REAPER Preferences->Appearance->Track Meters set meter update frequency to 8, meter decay to 12, meter minimum -62, meter max 6
- A send of .6 with these settings is 0 on the fader.
- For performance, you may want to change REAPER Preferences->Control/OSC/web to reduce update frequency.
- Use the Track Group view to set up VCAs and Mute Groups.

Setting Up Open Stage Control

- Create a directory to hold your Open Stage Control setup.
- For performance reasons, do not do this on the REAPER PC.
- Download Open Stage Control from here: https://openstagecontrol.ammd.net/

- Simply unzip it into the folder and it is ready to use in the unzipped location.
- In REAPER, go to Options->Preferences->Control/OSC/Web and click Add
    - For Control surface mode select OSC (Open Sound Control)
    - Enter a Device Name you can recognize as being for the REAPER OSC Interface
    - Under Pattern Config select "(open config directory)" and copy the ReaperOSC from this tool set available from WorkWebs.com to the config directory.
    - Back to REAPER and the Pattern Config dropdown, select "(refresh list)"
    - Then in the select oscfile that you just placed there
    - Set the Mode to Local Port
    - Set the Local Listen Port to a port number you want to use and that is open for traffic
    - At the bottom for Outgoing max packet size a recommendation is to use a packet size of 4096 and to wait 30 ms between packets. These settings slow the flow a bit to help reduce lost packets and overload.
- Set up a SEND in your Open Stage Control window after you open the app. Specify the IP of your REAPER solution and the port in the "send" area.
- Set 8080 or some other port in the port area.
- In the load line select the json file that was included with this package.
- In client options enter zoom=1 (to remind yourself that you can scale the app window later by changing the number)
- Once you have REAPER running, click the arrow / triangle in the top left to start the process.

Control Surface GUI – Secondary Screen

This control surface is similar to the primary one, but adds more tracks.

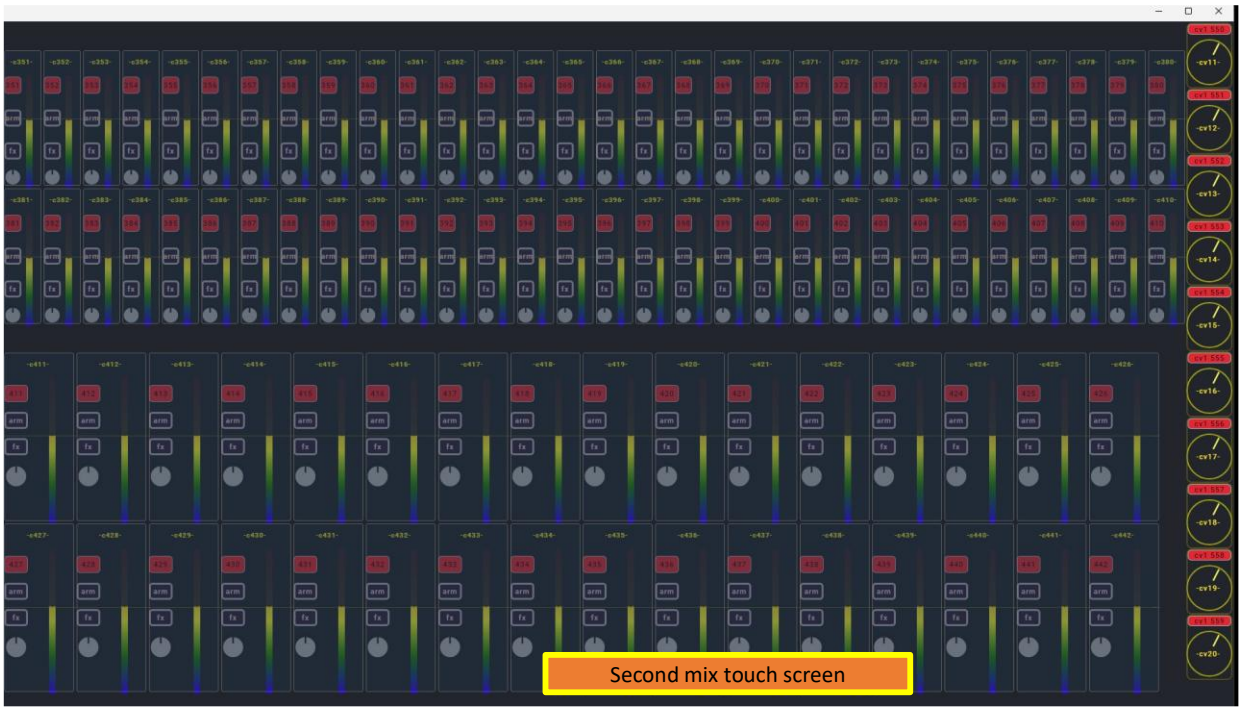


Figure 18 Secondary Control Surface

Other Tooling

Mixing Station 128 Head Amp Control for X32/M32

I have developed a Mixing Station layout I use to manage gains and Phantom Power for the 128 inputs that can have Head Amps in an x/m32 configuration (Local 1-32 + AES50 A 1-48 + AES50B 1-48 is 128 inputs). This is shared in the Mixing Station Community and is also included in this package as HAs….msz. The latest revision to Mixing Station seriously impacted colors but it still functions.

Figure 19 Mixing station x32/m32 Head Amp control for all 128 connections

WING 40 Channel Solo control

I have developed a Mixing Station layout I use to solos for a mic technician. This presents the 40 WING channels along with current mute status, a level meter to help isolate problem mics, channel names, and solo buttons. This allows a remote tech to solo channels and fix mic problems without requiring as much assistance from the main audio mixer. The headphone out is routed back into an inputon the wing and presented remotely for a headphone connection. I use an old analogue mixer and an direct out going to a local or AES50 output. This is shared as SOLO….msz.

Figure 20 Mixing station WING solo management

AutoHotKey Script

AutoHotKey ScriptIf you want hotkeys to run utility cues in this cue system, it is recommended that you try AutoHotKey software. A sample AHT file is included in this package.

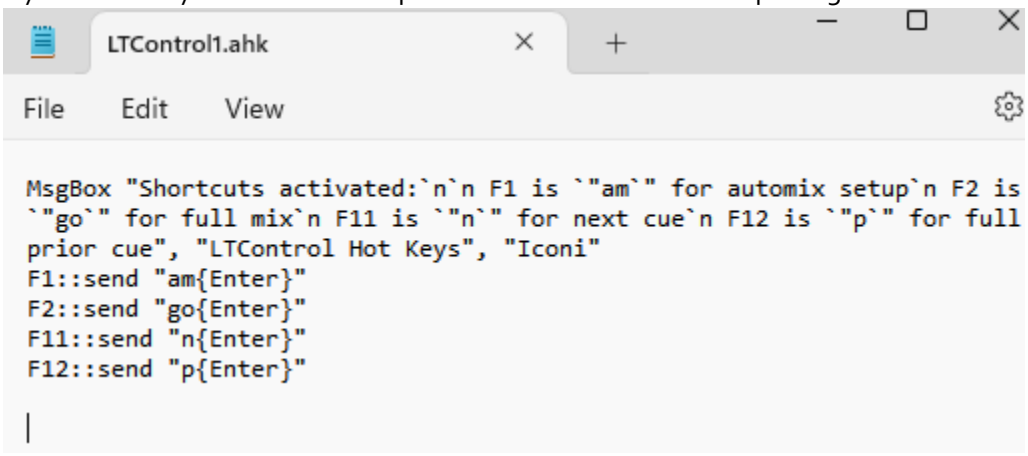


Figure 21 AutoHotKey sample script

Bitfocus Companion and Vicreo Listener

Bitfocus Companion and the Vicreo Listener can be used to provide a web interface to control the Cues. Run the Companion link with /tablet to see the simulated buttons.

Links

Download of **this document**, the **Track Muter, Track Muter and Volume setting** Web interface for REAPER**, Cue System and OSC command system**: https://workwebs.com

Facebook page for the Ecosystem in general and the WorkWebs tools: https://www.facebook.com/profile.php?id=61557113121231

TheatreMix by James Holt: https://theatremix.com/

Open Stage Control OSC interface builder/tool: https://openstagecontrol.ammd.net/

Fully Kiosk Browser for Android: https://www.fully-kiosk.com/

Paul Vannatto's OSC file processing tool: https://sourceforge.net/projects/lt-command/

AutoHotKey software for Windows: https://www.autohotkey.com/

SSL Solid State Logic plugin link: https://solidstatelogic.com/products/ssl-complete-bundle-subscription

WT Automixer for 16 tracks of Automixing in REAPER is available at: https://www.wtautomixer.com/

SSL 360 is available from the Support->Controllers link: https://support.solidstatelogic.com/hc/en-gb

REAPER: https://www.reaper.fm/download.php

DN9630 information: https://klarkteknik.com/product.html?modelCode=0606-ACT

General Behringer x32 information: https://www.behringer.com/catalog.html

Notepad++ powerful text editor for use on Windows: https://notepad-plus-plus.org/

Mixing Station: https://mixingstation.app/

Vicreo Listener: https://www.vicreo-listener.com/

Bitfocus Companion: https://bitfocus.io/companion

WING wosc and link to current OSC command list: https://sites.google.com/site/patrickmaillot/wing

VBAudio Matrix Coconut: To combine multiple ASIO compatible (and other) inputs together into a virtual interface under Windows: https://vb-audio.com/Matrix/coconut.htm